למידת מכונה

רשתות עצביות

Neural Networks

ייצוג מתמטי – סיווג בינארי

- אנו מחפשים קבוצת משקלים כך ש-

$$\begin{aligned}
&\operatorname{sgn}(w_0 x_{01} + w_1 x_{11} + w_2 x_{21} + \ldots\ldots + w_n x_{n1}) = y_1 \\
&\operatorname{sgn}(w_0 x_{02} + w_1 x_{12} + w_2 x_{22} + \ldots\ldots + w_n x_{n2}) = y_2 \\
&\ldots\ldots \\
&\ldots\ldots
\end{aligned}$$

$$\operatorname{sgn}\left(\left[w_0, w_1, w_2 \ldots\ldots w_n \right] \begin{bmatrix} x_{01} & x_{02} & x_{03} & \cdots & \cdots & \cdots & x_{0k} \\ x_{11} & x_{12} & x_{13} & \cdots & & & x_{1k} \\ x_{21} & x_{22} & x_{23} & \cdots & & & x_{2k} \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ x_{n1} & x_{n2} & x_{n3} & & & & x_{nk} \end{bmatrix} \right) = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ . \\ . \\ . \\ y_k \end{bmatrix}$$

הרעיון הכללי

לא תמיד יש הפרדה לינארית –
האסטרטגיה שלנו: נחפש את הפתרון שיביא אותנו
למינימום טעות על כל הדוגמאות

- נתחיל מקבוצת משקולות רנדומאליות
- נגדיר פונקצית טעות
- נחשב את הטעות הכללית (סכימה של כל הטעויות על כל דוגמא)
- נעדכן בכל שלב את המשקולות ע"מ להקטין את הטעות הכללית

$$w_{ij} \rightarrow w_{ij} + \Delta w_{ij}$$

- נשתמש באלגוריתם gradient descent

ננסה להבין להיכן להתקדם

$Loss(\theta_1, \theta_2)=(\theta_1 - 1)^2+3(\theta_2 - 3)^2$

Initial Guess: (θ_1= 2, θ_2=5) ⇒ Loss= 13

$Loss(\theta_1 + \epsilon, \theta_2) = Loss(2.00001,5) = 13.00002$

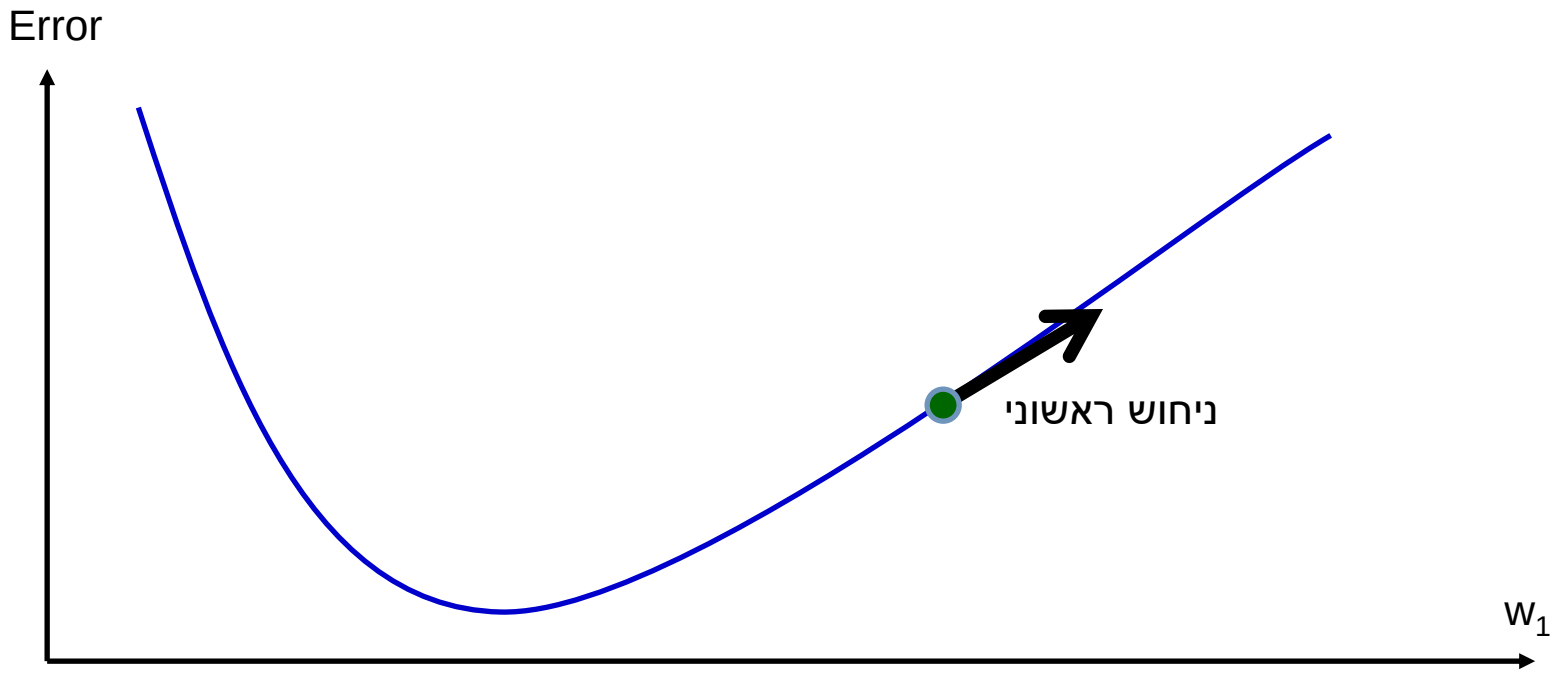
Decrease θ_1

$Loss(\theta_1, \theta_2 + \varepsilon) = Loss(2,5.00001) = 13.0001$

Decrease θ_2

Try a new Guess: (θ_1= 1.5, θ_2=4) ⇒ Loss= 3.25

משתנה אחד – Gradient descent



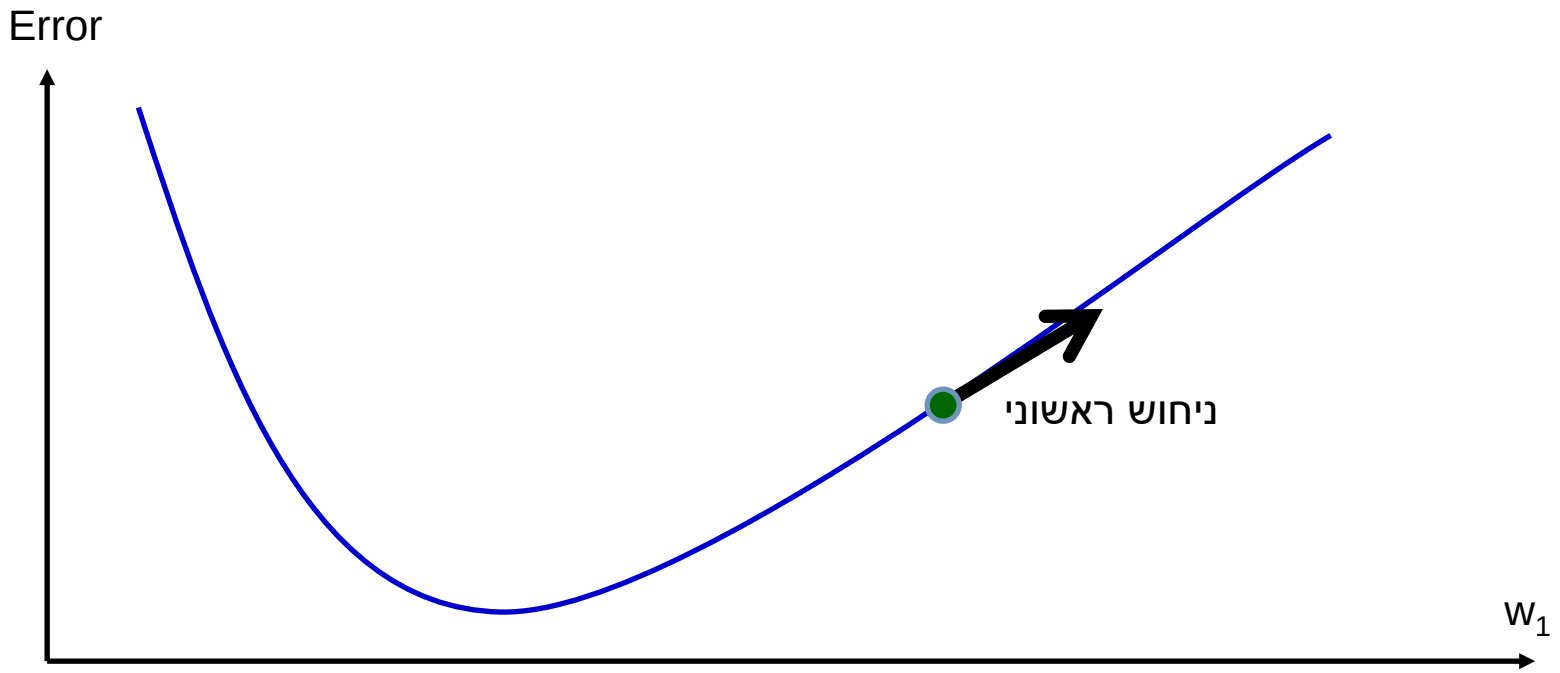
דרך מתוחכמת יותר – Gradient Descent

- **נניח שיש לנו את פונקציית הטעות E כפונציה של המשקל w:** **E = f(w)**
- **א':** כאשר $\frac{dE}{dw} = 0$ אנו נמצאים בנקודת מינימום או מקסימום
- **ב':** כאשר $\frac{dE}{dw} > 0$ אזי f(w) גדל ככל ש-w גדל
- **ג':** כאשר $\frac{dE}{dw} < 0$ אזי f(w) קטן ככל ש-w גדל

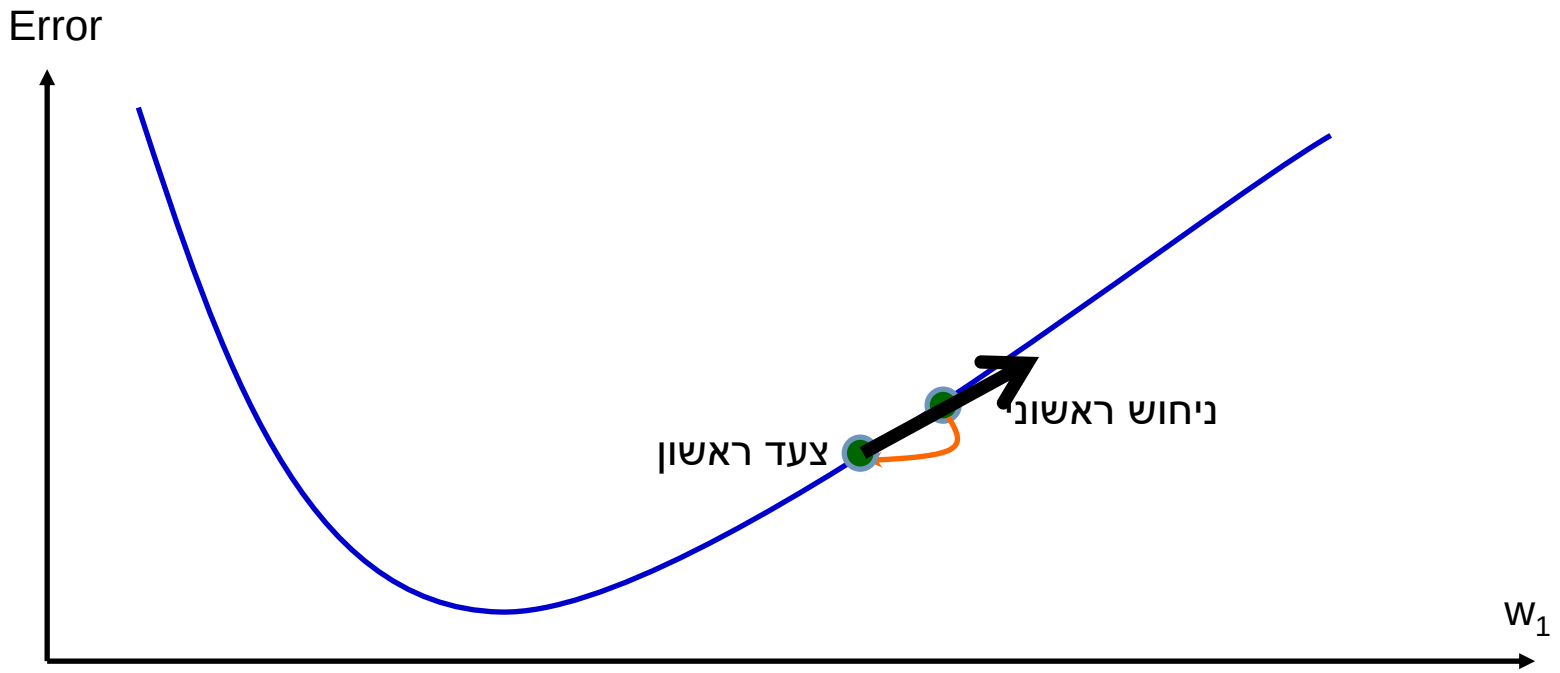
- *E* הינה פונקציית הטעות אותה נרצה להקטין, לכן:
  - עבור מקרה **ב'** – נרצה להקטין את w
  - עבור מקרה **ג'** – נרצה להגדיל את w
  - צריך ללכת בכיוון הפוך לעוצמת הגרדיאנט ←

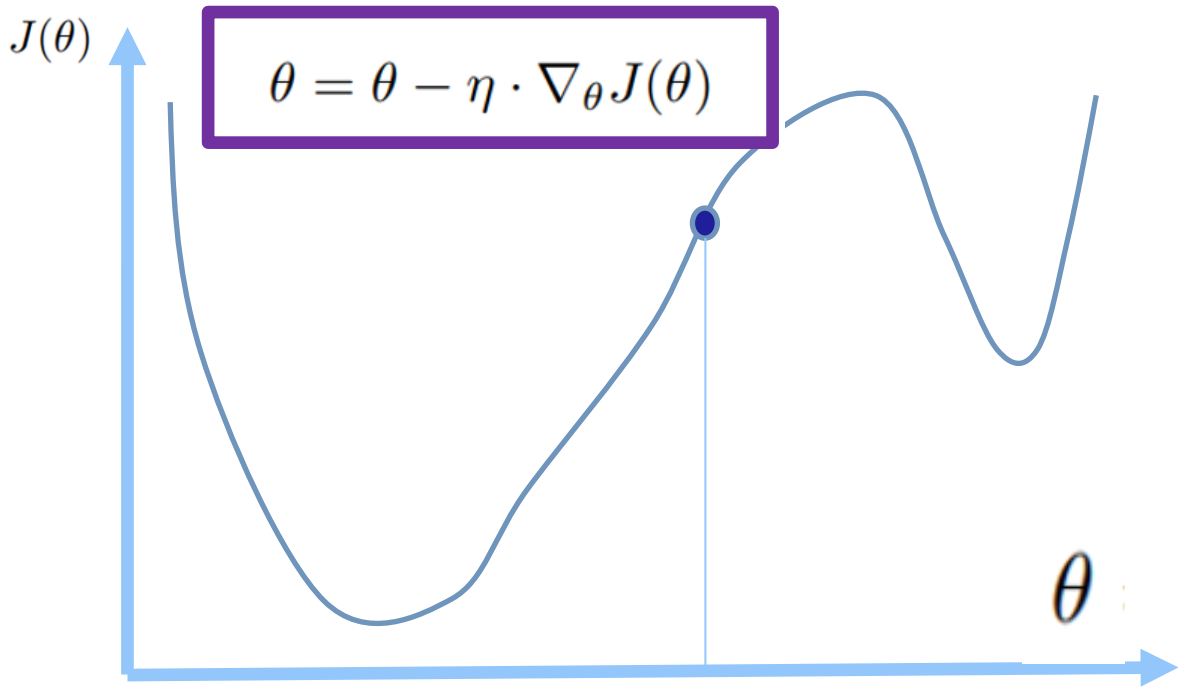
$$\Delta w = -\eta \frac{\partial E}{\partial w}$$

משתנה אחד – Gradient descent

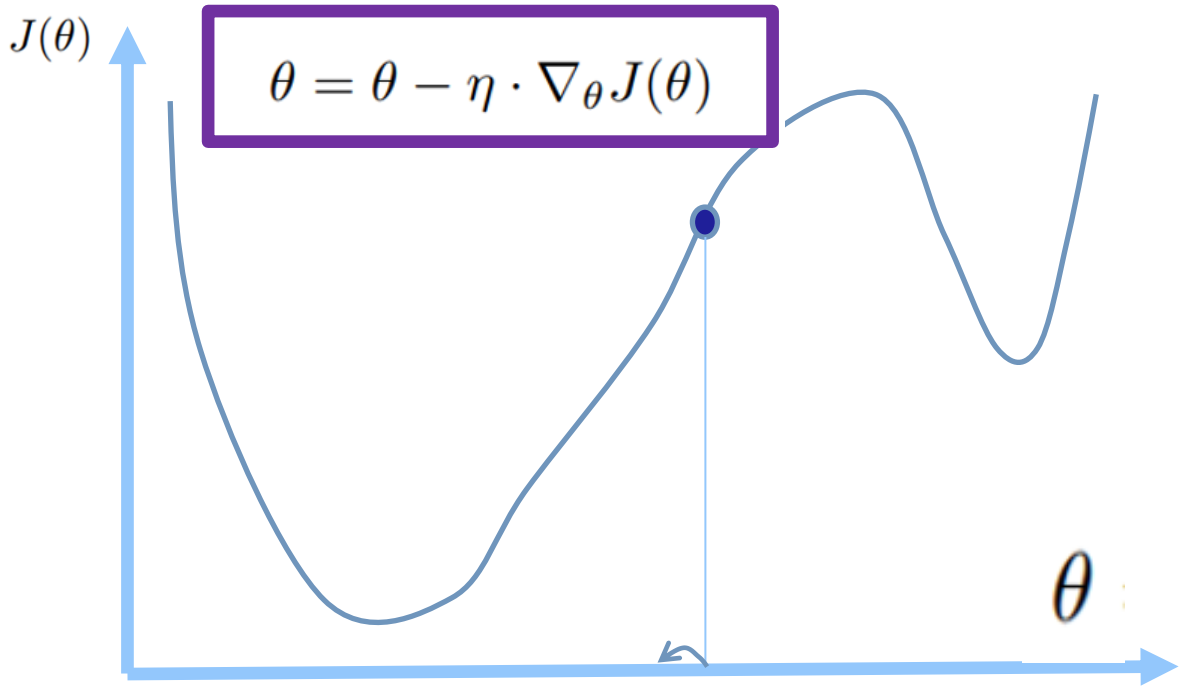


משתנה אחד – Gradient descent

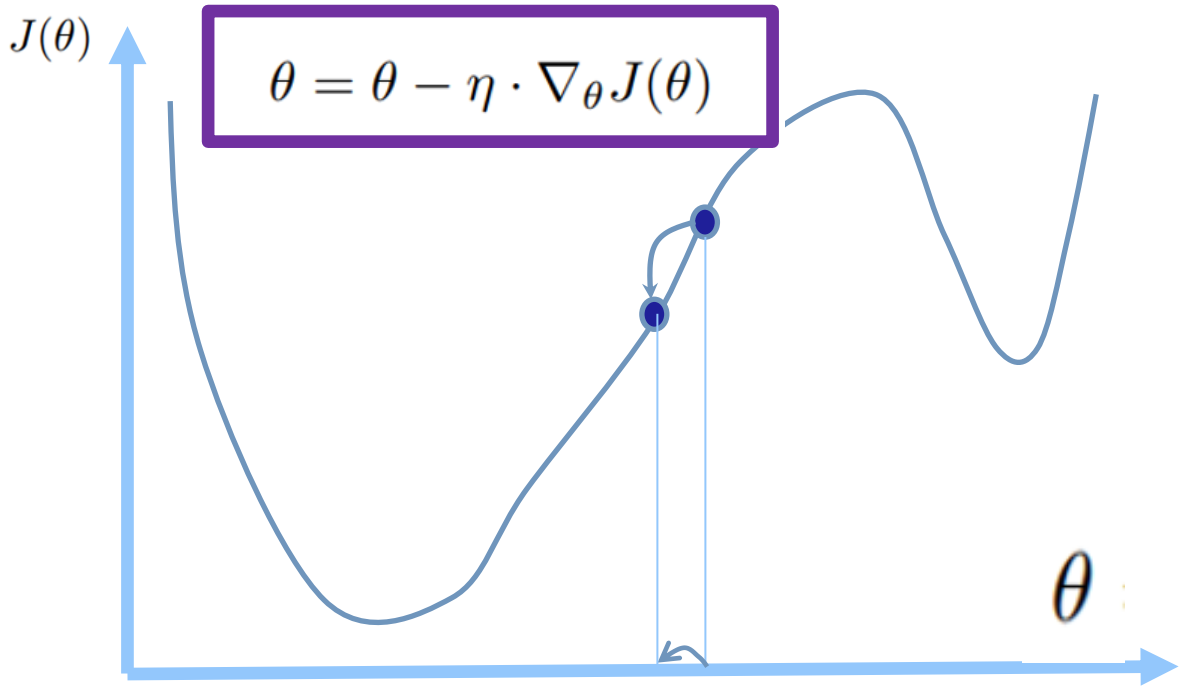




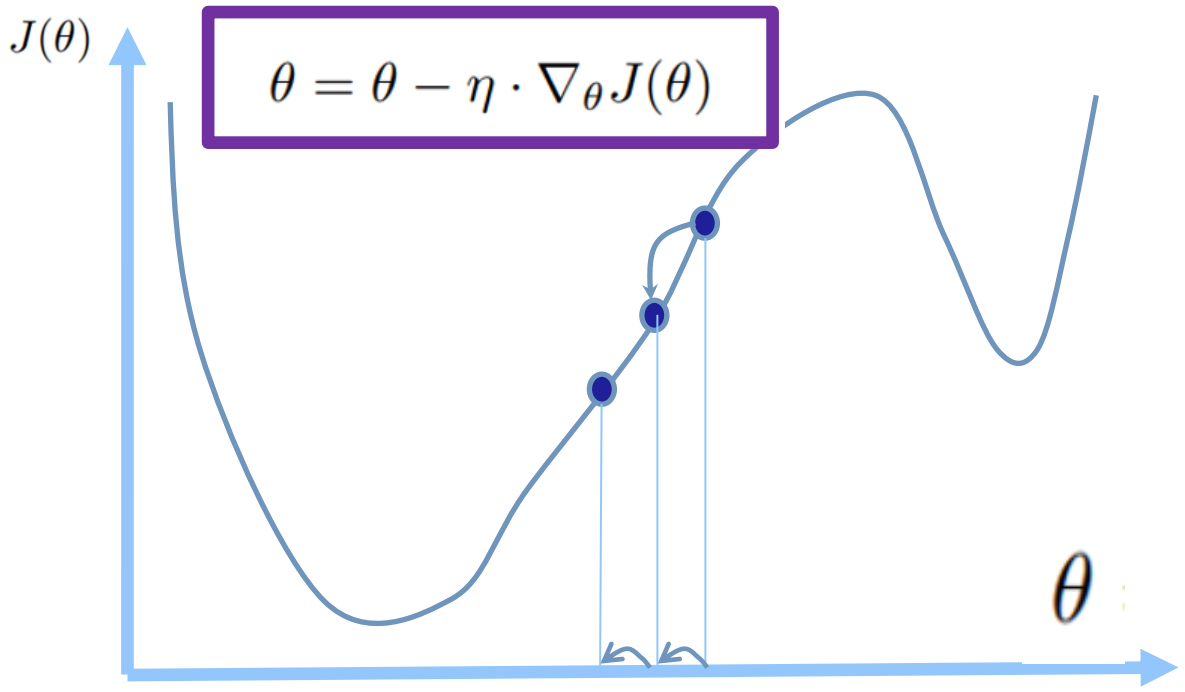
$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$



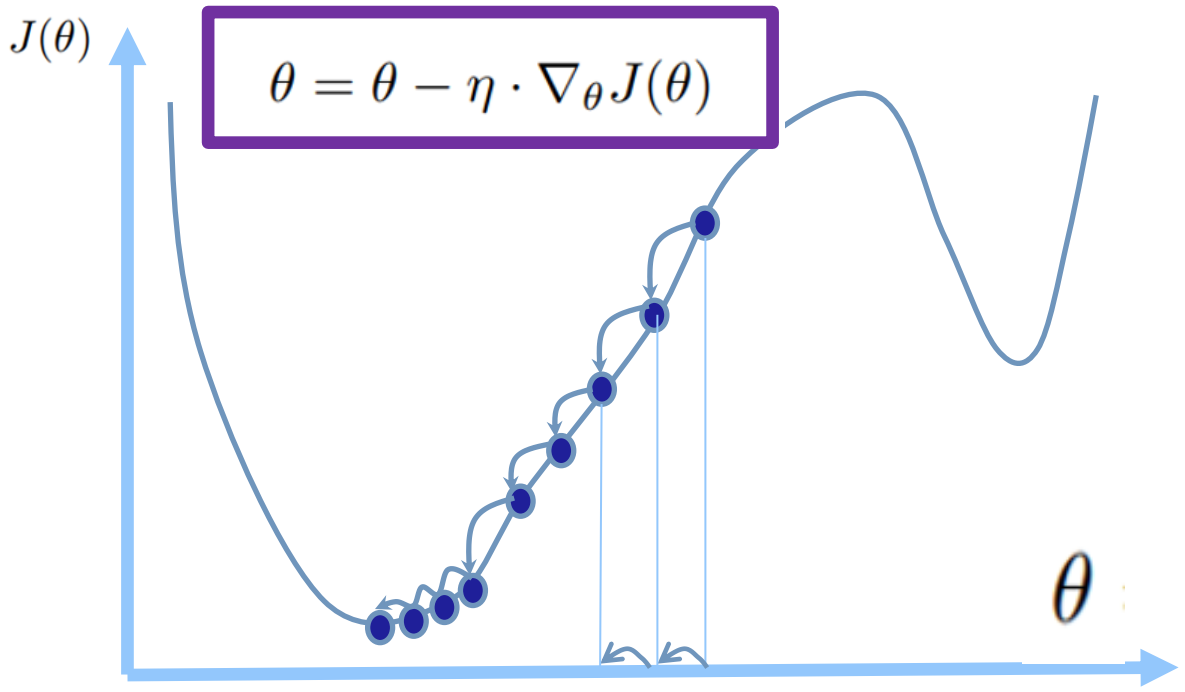
$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$



$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

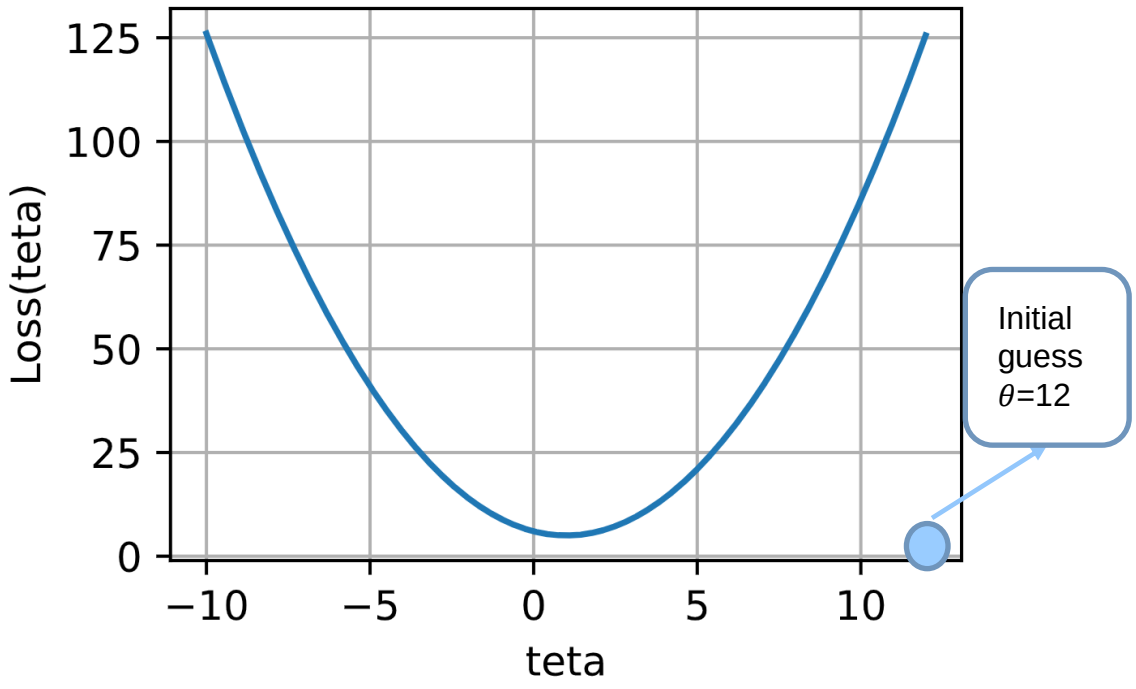


$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

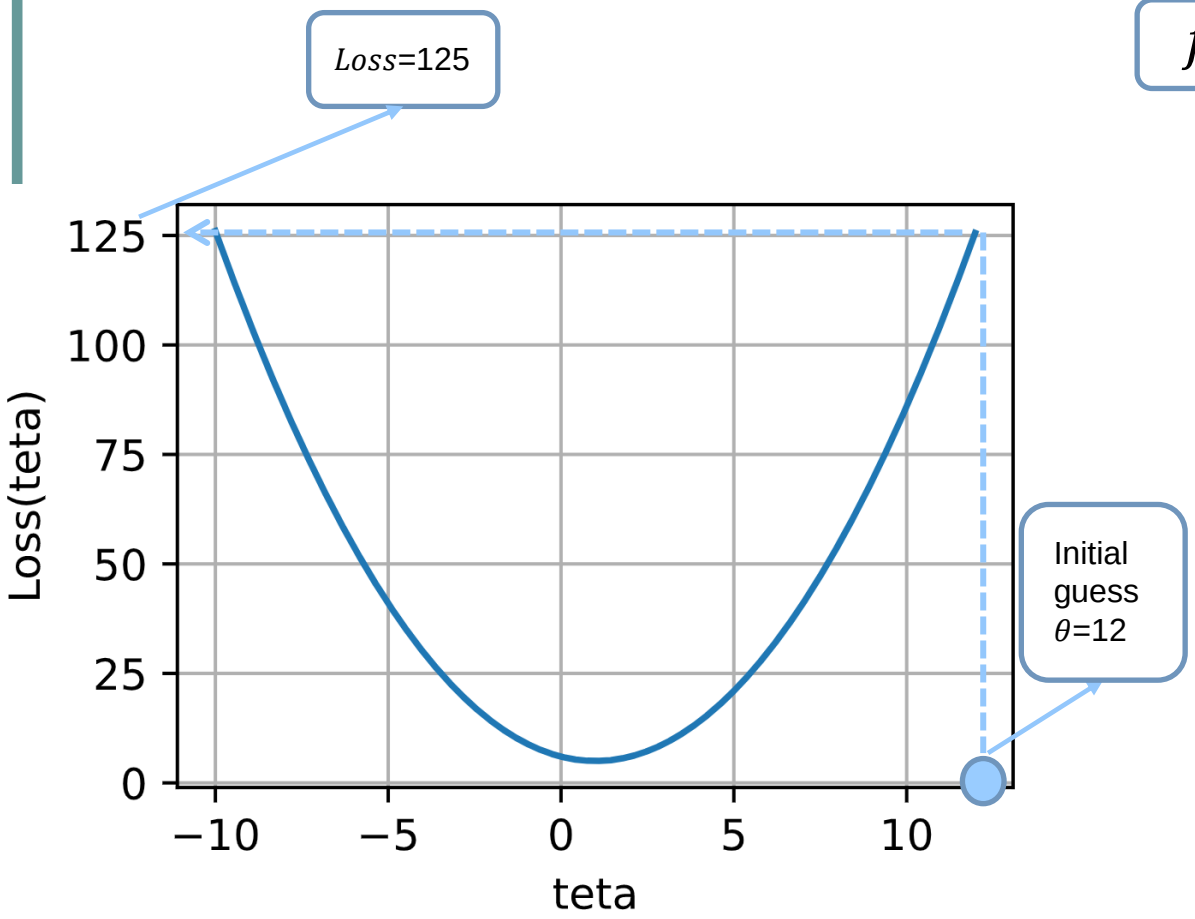


$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

$$f(\theta) = (\theta - 1)^2 + 5$$

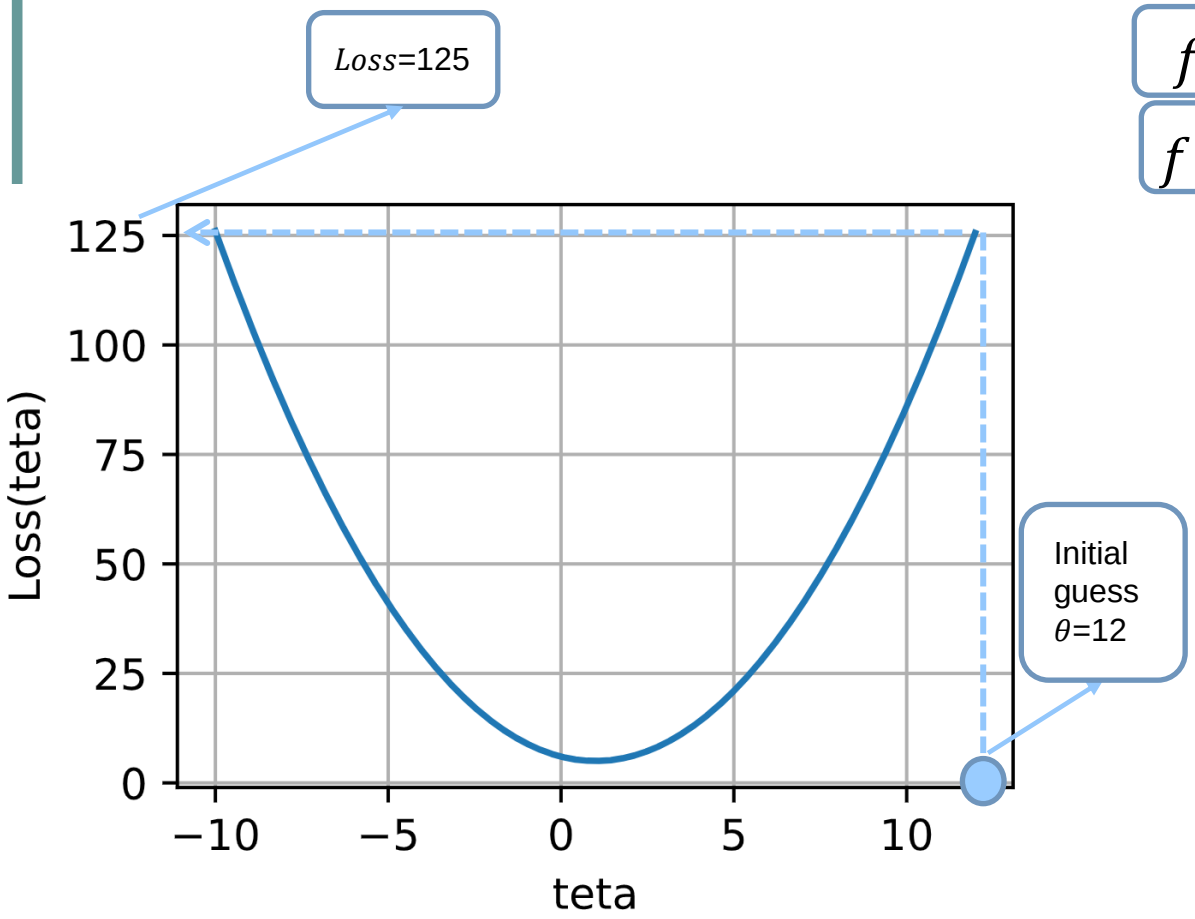


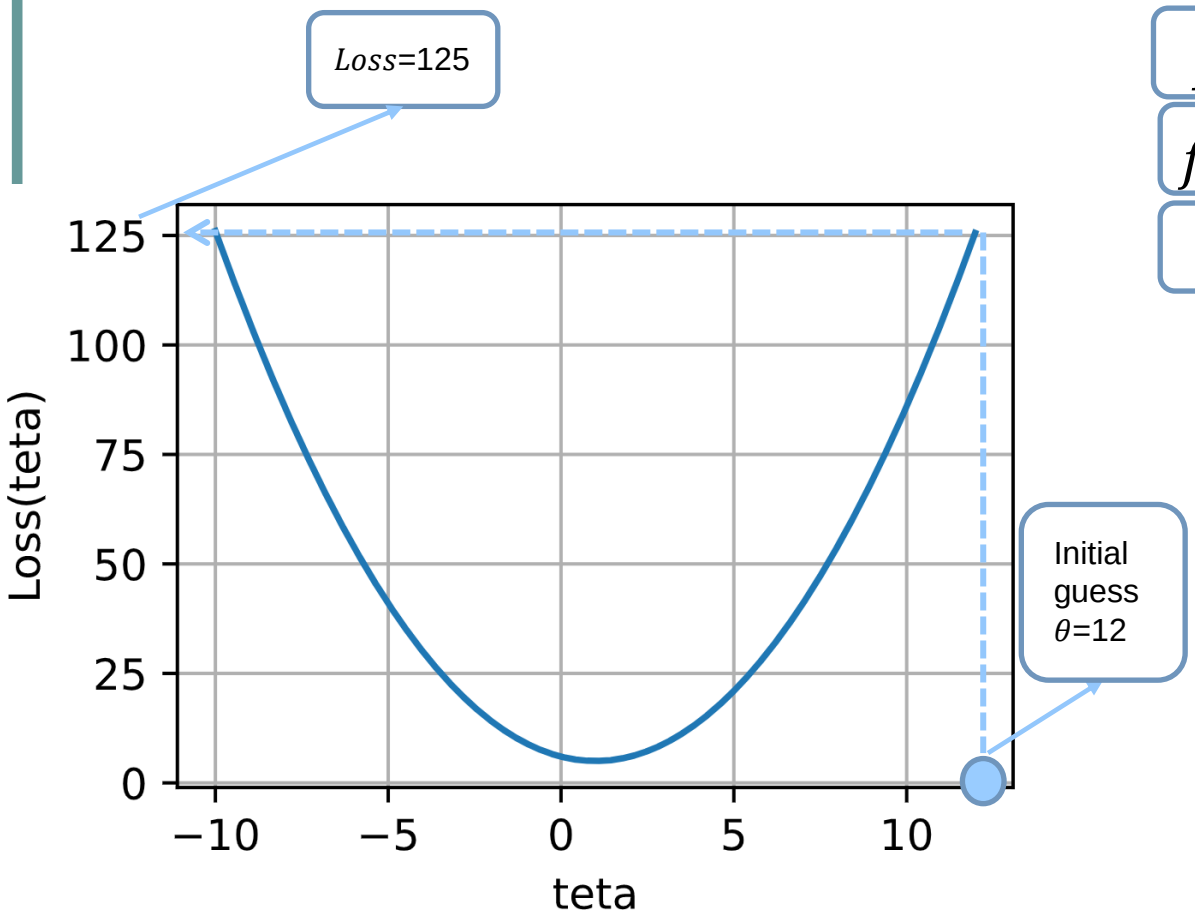
$f(\theta) = (\theta - 1)^2 + 5$



$f(\theta) = (\theta - 1)^2 + 5$

$f'(\theta) = 2(\theta - 1)$

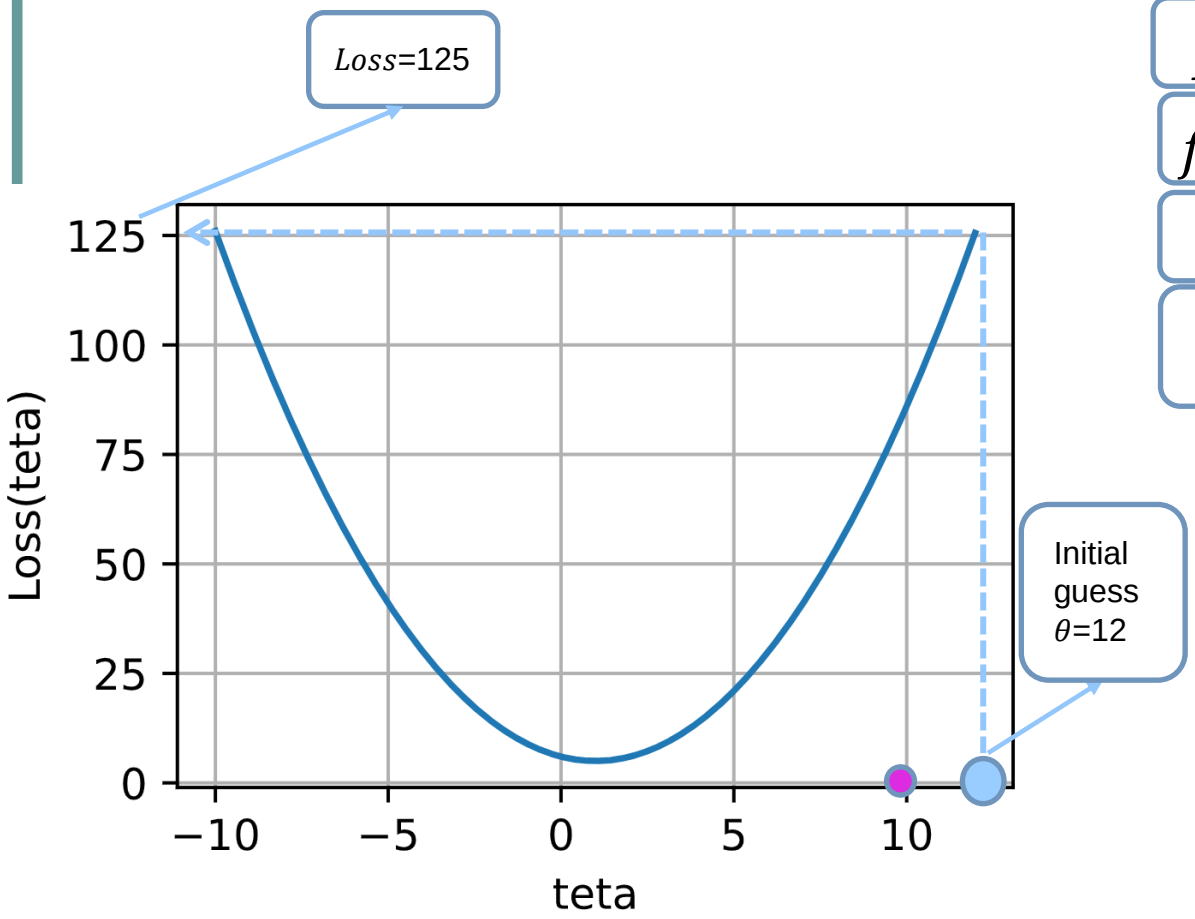




$$f(\theta) = (\theta - 1)^2 + 5$$

$$f'(\theta) = 2(\theta - 1)$$

$$f'(\theta = 12) = 2(12 - 1) = 22 > 0$$



$$f(\theta) = (\theta - 1)^2 + 5$$

$$f'(\theta) = 2(\theta - 1)$$

$$f'(\theta = 12) = 2(12 - 1) = 22 > 0$$

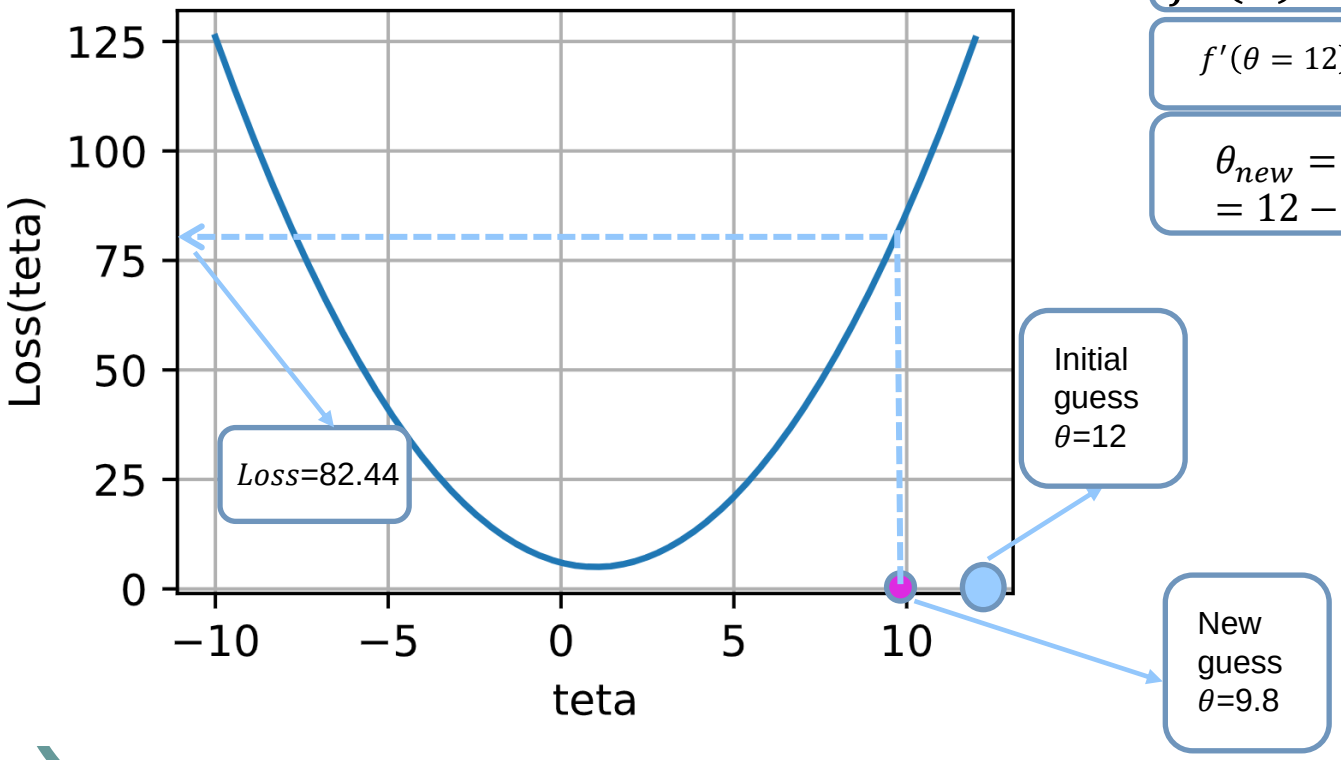
$$\theta_{new} = \theta_{old} - \eta f'(\theta = 12)$$
$$= 12 - 0.1 \times 22 = 9.8$$

$$f(\theta) = (\theta - 1)^2 + 5$$

$$f'(\theta) = 2(\theta - 1)$$

$$f'(\theta = 12) = 2(12 - 1) = 22 > 0$$

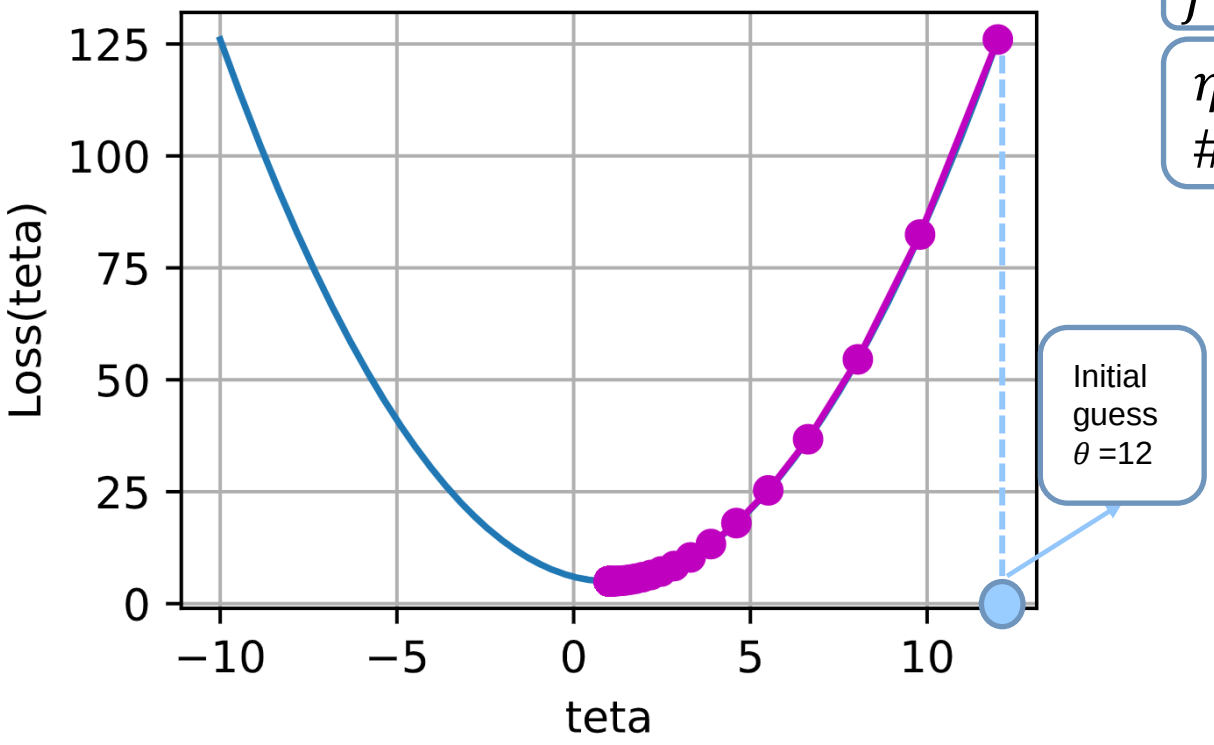
$$\theta_{new} = \theta_{old} - \eta f'(\theta = 12)$$
$$= 12 - 0.1 \times 22 = 9.8$$



$f(\theta) = (\theta - 1)^2 + 5$

$f'(\theta) = 2(\theta - 1)$

η =0.1
#of iterations = 200

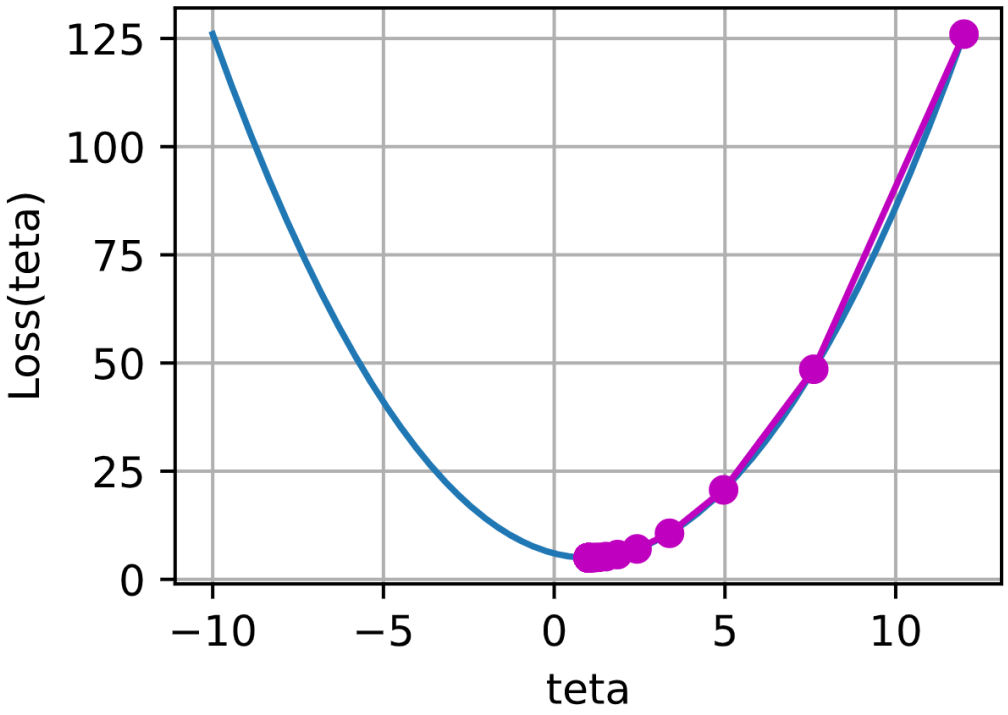


$f(\theta) = (\theta - 1)^2 + 5$

$f'(\theta) = 2(\theta - 1)$

η =0.2

#of iterations = 200

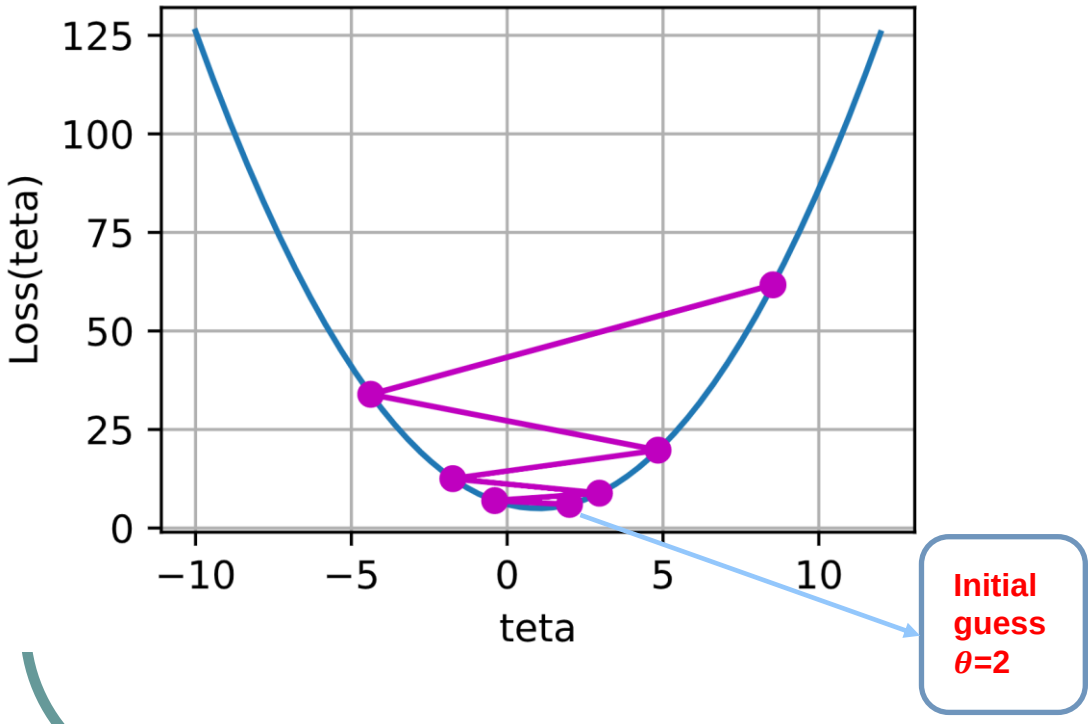


$f(\theta) = (\theta - 1)^2 + 5$

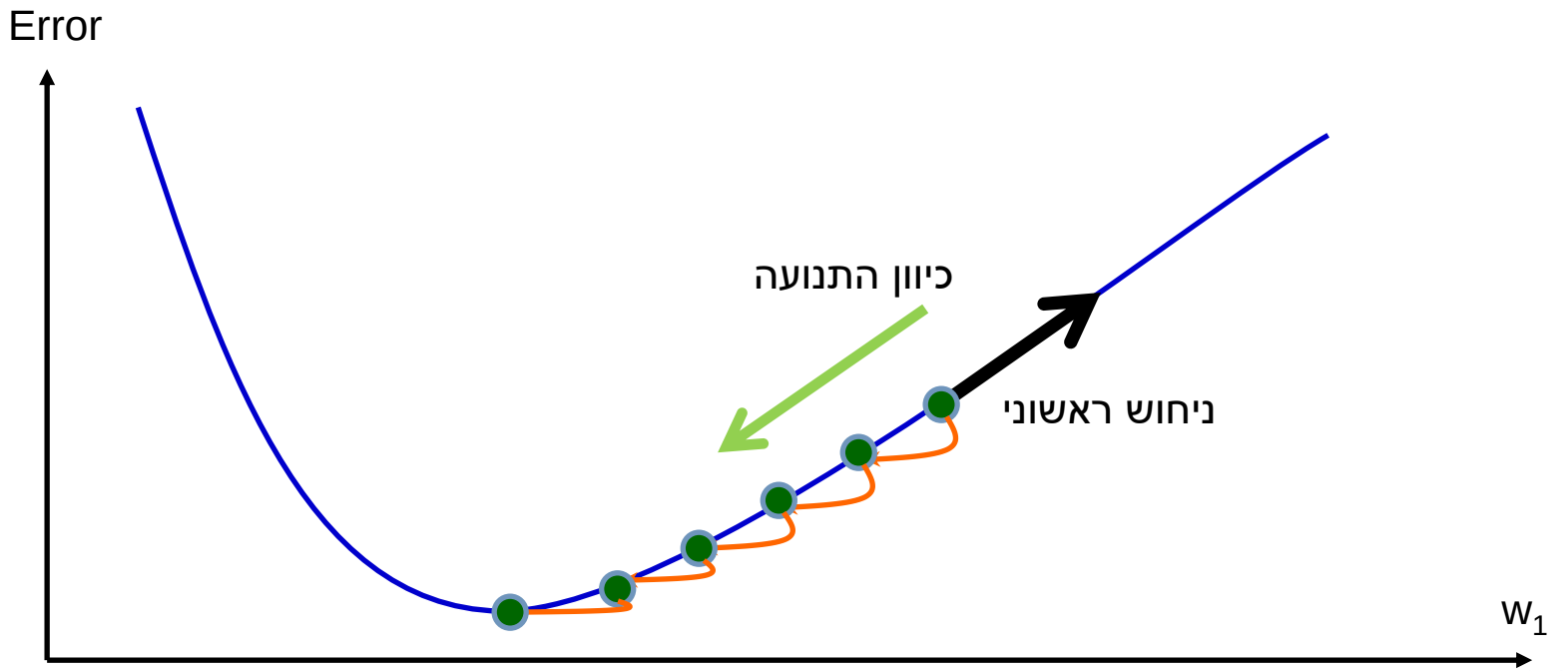
$f'(\theta) = 2(\theta - 1)$

η =**1.2**

#of iterations = 5



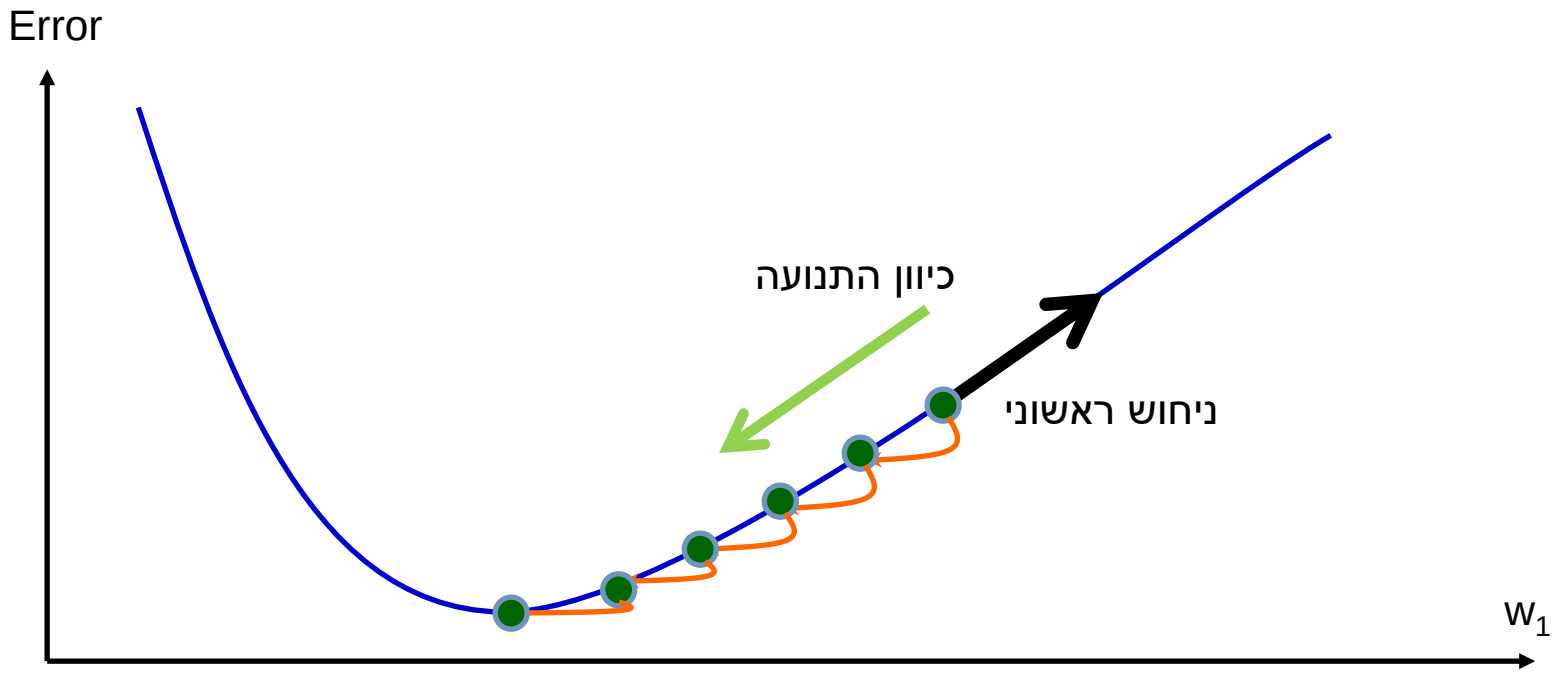
משתנה אחד – Gradient descent



הרעיון:

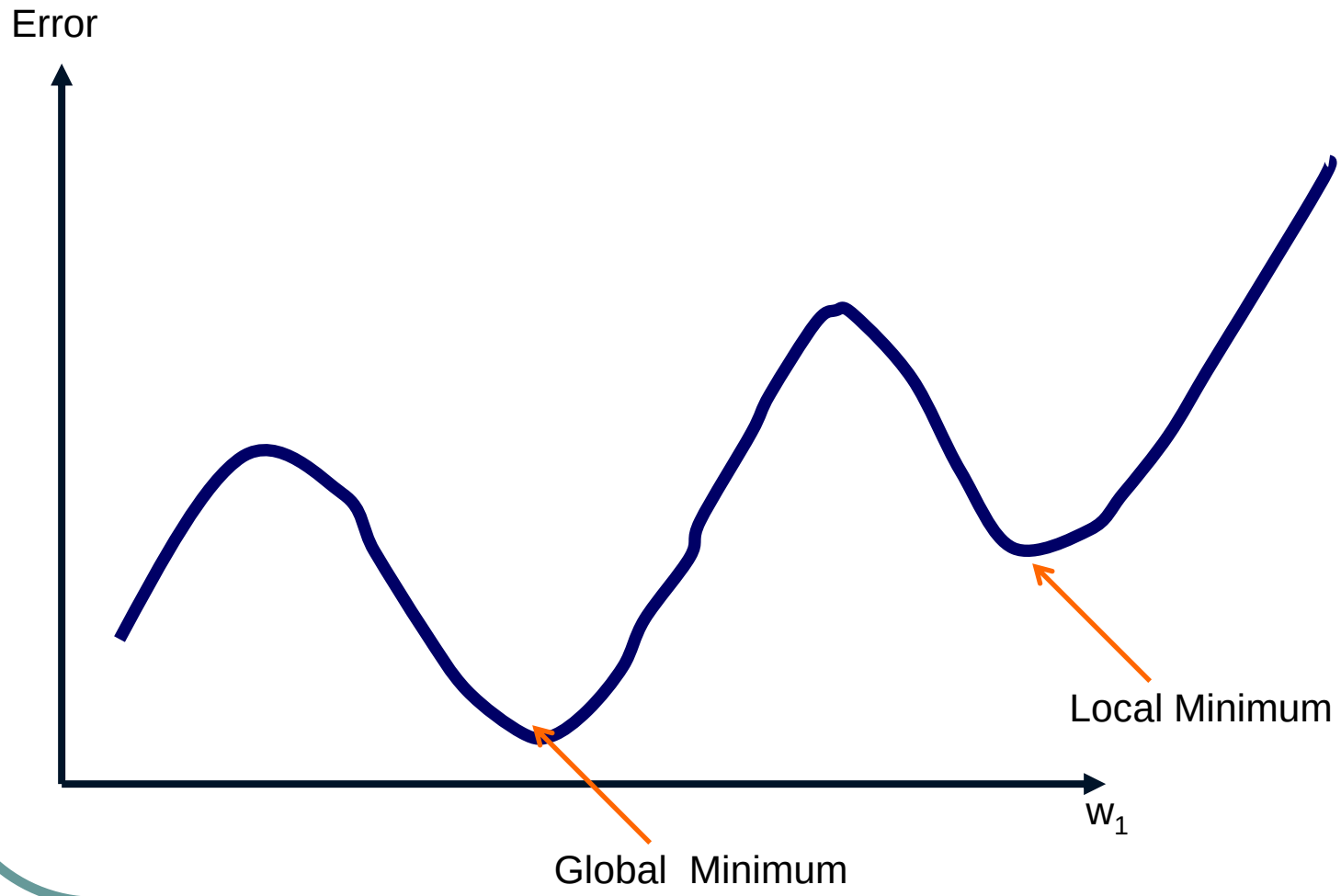
1. מצא את כיוון הגרדיאנט
2. חשב כיוון תנועה עפ"י 1
3. הגדר "צעד"
4. נוע בכיוון הצעד
5. חזור על סעיפים 1-4 עד שמתקבל ערך טעות הקטן מספיק

משתנה אחד – Gradient descent



צעד קטן מדי – לא יעיל מבחינה חישובית
צעד גדול מדי – עלול להביא לתוצאה לא נכונה

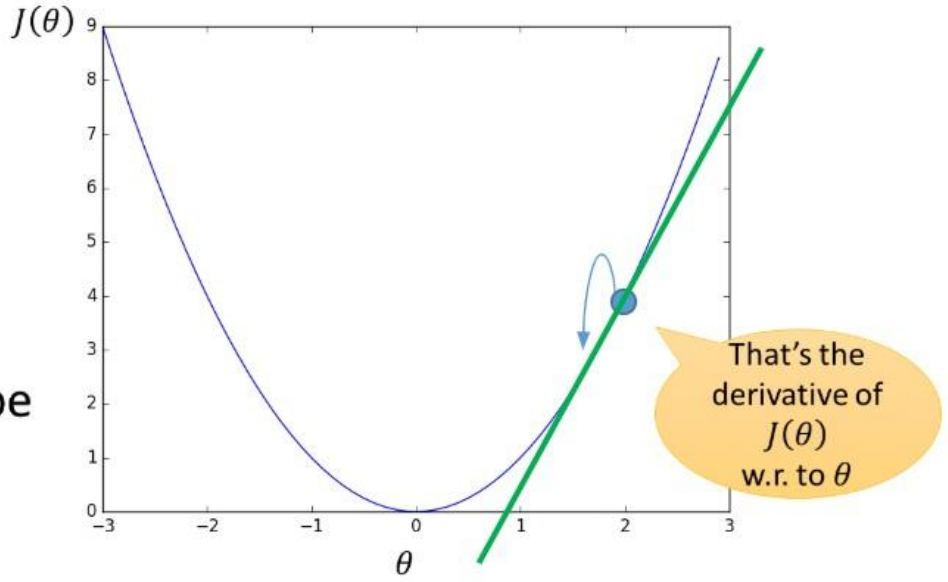
Local minima vs. Global minimum



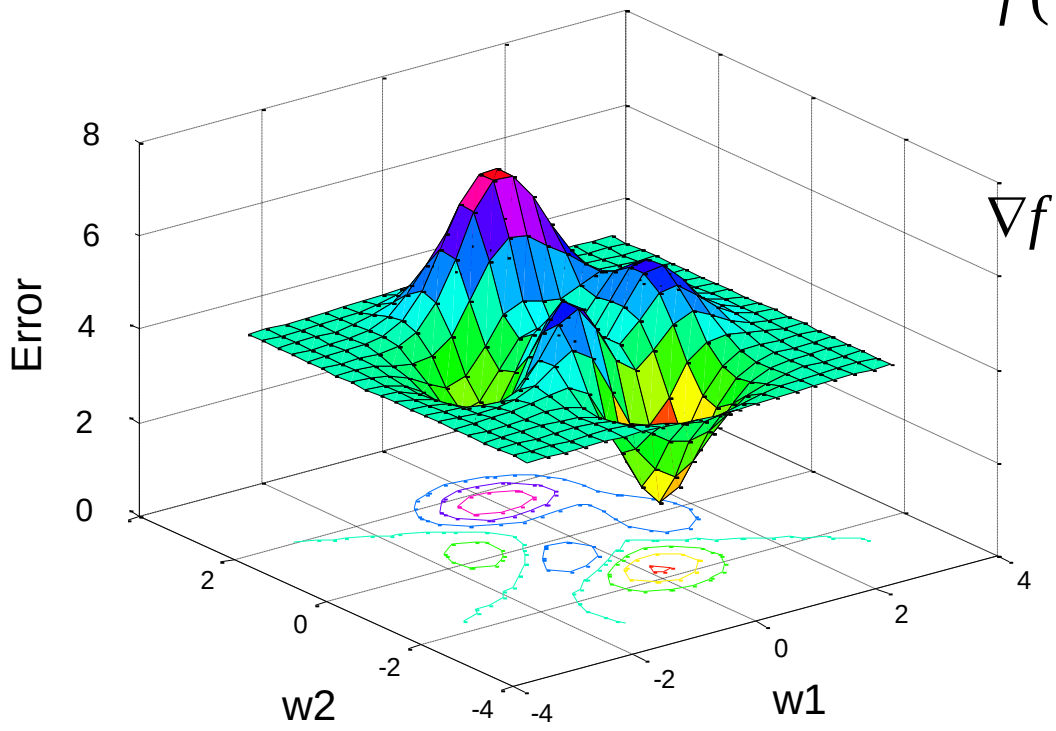
סיכום עד כה

Training as a minimization process

- Given a training set (dataset for finding the optimal model parameters), we usually minimize the **train-loss**, that is, the loss over the training set
- We use **Gradient Descent**
- We start with random values for θ, and then update them to minimize $J(\theta)$
- Update is done using the **opposite direction** of the slope of $J(\theta)$ at θ



בשני מימדים



$$f(w):\Re^2 \to \Re$$

$$\nabla f(w_1, w_2) = \begin{pmatrix} \frac{\partial}{\partial w_1} Error(w) \\ \frac{\partial}{\partial w_2} Error(w) \end{pmatrix}$$

נגזרות חלקיות וגרדיאנט

- **נגזרת חלקית** של פונקציה בכמה משתנים $\frac{\partial f}{\partial x_k}$ היא נגזרת של הפונקציה באחד ממשתניה, כאשר מתייחסים לשאר המשתנים כאל קבועים

- גרדיאנט – ווקטור הנגזרות החלקיות

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots\ldots, \frac{\partial f}{\partial x_k} \right)$$

Gradient descent in *m* dimensions

Given $f(\mathbf{w}) : \Re^m \rightarrow \Re$

$$\nabla f(w) = \begin{pmatrix} \frac{\partial}{\partial w_1} f(w) \\ \vdots \\ \frac{\partial}{\partial w_m} f(w) \end{pmatrix}$$ points in direction of steepest ascent.

GRADIENT DESCENT RULE: $w \leftarrow w - \eta \nabla f(w)$

Equivalently $w_j \leftarrow w_j - \eta \frac{\partial}{\partial w_j} f(w)$ ....where w_j is the jth weight

"just like a linear feedback system"

http://www.autonlab.org/tutorials/

$E = f(w)$

m – מס' המאפיינים המייצגים דוגמא $(x_1, x_2, \ldots\ldots, x_m)$

R – מס' הדוגמאות בקבוצת האימון

$\sum_{i=0}^{m} w_i x_i$

$$E = \sum_{i=1}^{R} (Prediction_i - y_i)^2$$

$$E = \sum_{i=1}^{R} (\sum_{i=0}^{m} w_i x_i - y_i)^2$$

$$\frac{\partial E}{\partial w_i} = ?$$

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

$$= \sum_{k=1}^{R} 2(y_k - \mathbf{w}^T \mathbf{x}_k) \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

$$= \sum_{k=1}^{R} 2(y_k - \mathbf{w}^T \mathbf{x}_k) \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)$$

$$= -2 \sum_{k=1}^{R} \delta_k \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}_k$$

...where...

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

$$= \sum_{k=1}^{R} 2(y_k - \mathbf{w}^T \mathbf{x}_k) \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)$$

$$= -2 \sum_{k=1}^{R} \delta_k \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}_k$$

...where...

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

$$= -2 \sum_{k=1}^{R} \delta_k \frac{\partial}{\partial w_j} \sum_{i=1}^{m} w_i x_{ki}$$

$$= -2 \sum_{k=1}^{R} \delta_k x_{kj}$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

R – מס' הדוגמאות בקבוצת האימון

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

...where...

$$\frac{\partial E}{\partial w_j} = -2 \sum_{k=1}^{R} \delta_k x_{kj}$$

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

We frequently neglect the 2 (meaning we halve the learning rate)

http://www.autonlab.org/tutorials/

The “batch” perceptron algorithm

1) Randomly initialize weights w_1 w_2 … w_m

2) Get your dataset (append 1’s to the inputs if you don’t want to go through the origin).

3) for $i = 1$ to R $\quad \delta_i := y_i - \mathbf{w}^T \mathbf{x}_i$

4) for $j = 1$ to m $\quad w_j \leftarrow w_j + \eta \sum_{i=1}^{R} \delta_i x_{ij}$

5) if $\sum \delta_i^2$ stops improving then stop. Else loop back to 3.

http://www.autonlab.org/tutorials/

דוגמא

AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



AND			
X0	X1	X2	Y
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

דוגמא

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

AND			
X0	X1	X2	Y
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Iteration 0: w0 = 0.3, w1=0.4, w2=0.5 – ניחוש ראשוני

AND				out	Error
X0	X1	X2	Y		
1	0	0	0	0.3	δ1 = -0.3
1	0	1	0	0.8	δ2 = -0.8
1	1	0	0	0.7	δ3 = -0.7
1	1	1	1	1.2	δ4 = -0.2

מעבר על כל הדוגמאות וחישוב הטעות

$$E = (-0.3)^2 + (-0.8)^2 + (-0.7)^2 + (-0.2)^2 = 0.09$$

דוגמא

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

Iteration 0: w0 = 0.3, w1=0.4, w2=0.5

AND				out		Error
X0	X1	X2	Y			
1	0	0	0	0.3		δ1 = -0.3
1	0	1	0	0.8		δ2 = -0.8
1	1	0	0	0.7		δ3 = -0.7
1	1	1	1	1.2		δ4 = -0.2

$$\eta = 0.05$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k0} = 0.1 \times (-0.3 \times 1 + (-0.8) \times 1 + (-0.7) \times 1 + (-0.2 \times 1) = -0.2$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k1} = 0.1 \times (-0.3 \times 0 + (-0.8) \times 0 + (-0.7) \times 1 + (-0.2 \times 1) = -0.09$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k2} = 0.1 \times (-0.3 \times 0 + -0.8 \times 1 + -0.7 \times 0 + (-0.2 \times 1) = -0.1$$

דוגמא-AND

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

Iteration 0: w0 = 0.3, w1=0.4, w2=0.5

AND				out		Error
X0	X1	X2	Y			
1	0	0	0	0.3		
1	0	1	0	0.8		
1	1	0	0	0.7		
1	1	1	1	1.2		

$$2\eta \sum_{k=1}^{4} \delta_k x_{k0} = -0.2$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k1} = -0.09$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k2} = -0.1$$

$$w_0 = w_0 + \Delta w_0 = 0.3 + (-0.2) = 0.1$$

$$w_1 = w_1 + \Delta w_1 = 0.4 + (-0.09) = 0.31$$

$$w_2 = w_2 + \Delta w_2 = 0.5 + (-0.1) = 0.4$$

דוגמא-AND

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

Iteration 1: w0 = 0.1, w1=0.31, w2=0.4

AND				out	Error
X0	X1	X2	Y		
1	0	0	0	0.1	$\delta 1 = -0.1$
1	0	1	0	0.5	$\delta 2 = -0.5$
1	1	0	0	0.41	$\delta 3 = -0.41$
1	1	1	1	0.81	$\delta 4 = -0.19$

$$E = (0.1)^2 + (0.5)^2 + (0.41)^2 + (0.19)^2 = 0.0361$$

$$0.0361 < 0.09$$

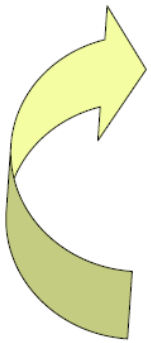
נמשיך לעוד איטרציה.....

We can use the algorithm in incremental mode

קיימות מס 'צורות הפעלה לאלגוריתם:

1) מוד Gradient Descent GD (מכונה לעיתים Batch)
2) מוד Stochastic Gradient Descent SGD (מכונה לעיתים Increnmetal)
3) מוד Mini-Batches

SGD Mode



observe $(\mathbf{x}, y)$

$$\delta \leftarrow y - \mathrm{w}^{\mathrm{T}}\mathrm{x}$$

$$\forall j \;\; w_j \leftarrow w_j + \eta\, \delta\, x_j$$

http://www.autonlab.org/tutorials/

Gradient Descent (GD)

- Gradient descent - update weights only after accumulating gradients for all training-set instances

```
for e in range(epoch_number):
        for x,y in training set:
                Calculate Delta
                Update Weights
```

$O(md)$

$O(d)$

Stochastic Gradient Descent (SGD)

- **Stochastic** Gradient descent - update weights after every instance. Usually - shuffle the training set before each epoch

```
for e in range(epoch_number):
        shuffle(training_set)
        for x,y in training_set:
            Update Weights
```

O(m)

O(md)

For classification

m – מס' המאפיינים המייצגים דוגמא $(x_1, x_2, \ldots, x_m)$

R – מס' הדוגמאות בקבוצת האימון

$$\boldsymbol{SGN} \sum_{i=0}^{m} w_i x_i$$

$$E = \sum_{i=1}^{R} \left(Prediction_i - y_i \right)^2$$

$$E = \sum_{i=1}^{R} \left(\boldsymbol{SGN} \left(\sum_{i=0}^{m} w_i x_i \right) - y_i \right)^2$$

$$\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}} = ?$$

Update Rule – Classification, SGD

If wrongly predicted as 1 $(y_i = -1, h(x) = 1)$

$$w \leftarrow w - x_i$$

If wrongly predicted as -1 $(y_i = 1, h(x) = -1)$

$$w \leftarrow w + x_i$$

If predicted correct – Do not change the weights

דוגמא SGD

- דוגמא:
  - חישוב משקלי פרספטרון
  - מוד אינקרמנטלי
  - נתחיל ממשקלים אקראיים: w_0=-0.5, w_1 = 2, w_2=4

AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



AND			
X0	X1	X2	Y
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

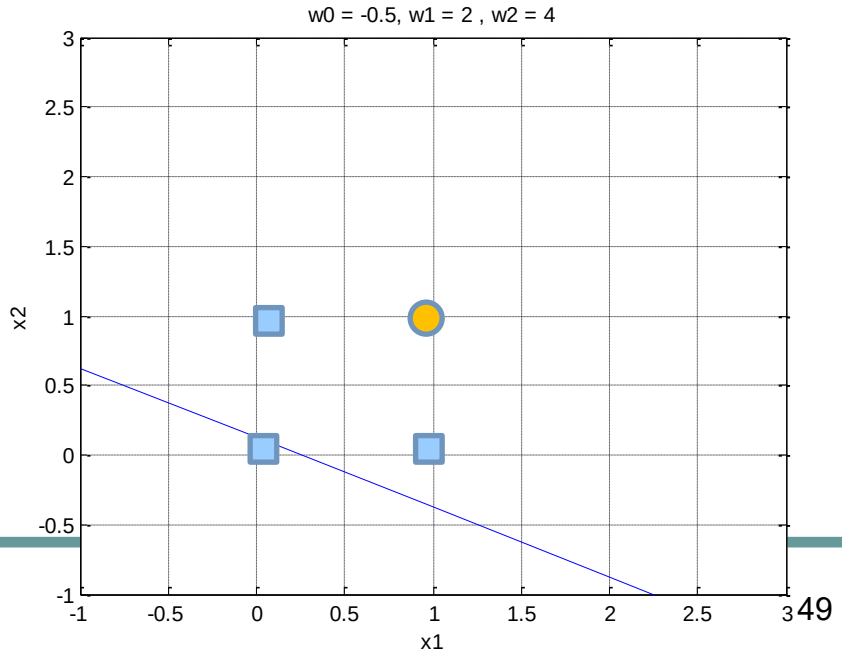
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

דוגמא SGD

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no



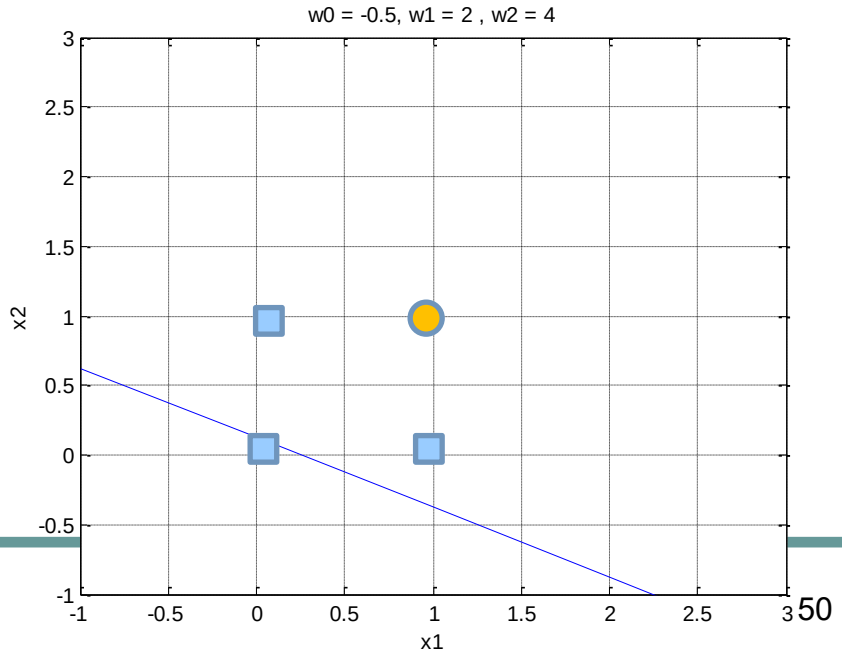
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no
1	0	1	0	-0.5	2	4	3.5	POS	yes



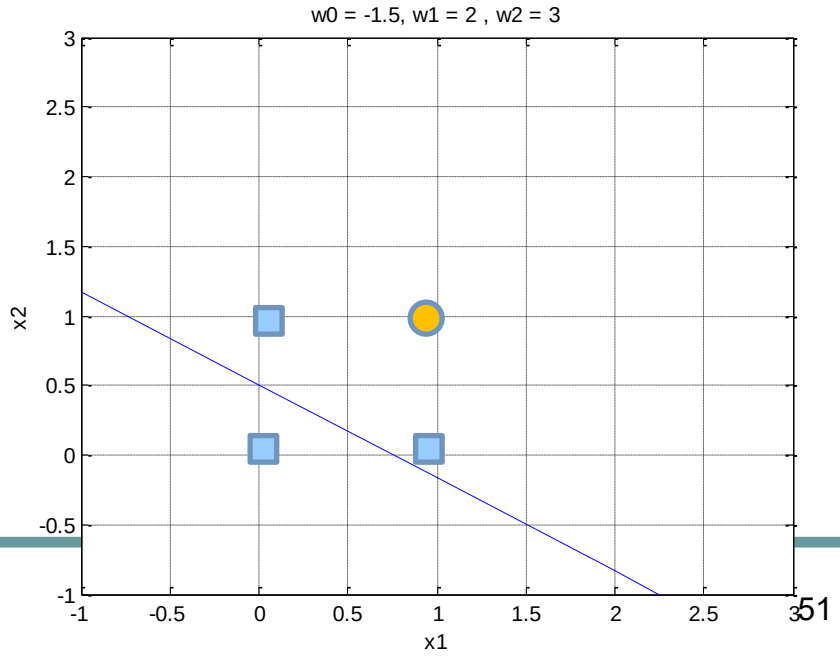
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no
1	0	1	0	-0.5	2	4	3.5	POS	yes
1	1	0	0	-1.5	2	3	0.5	POS	yes



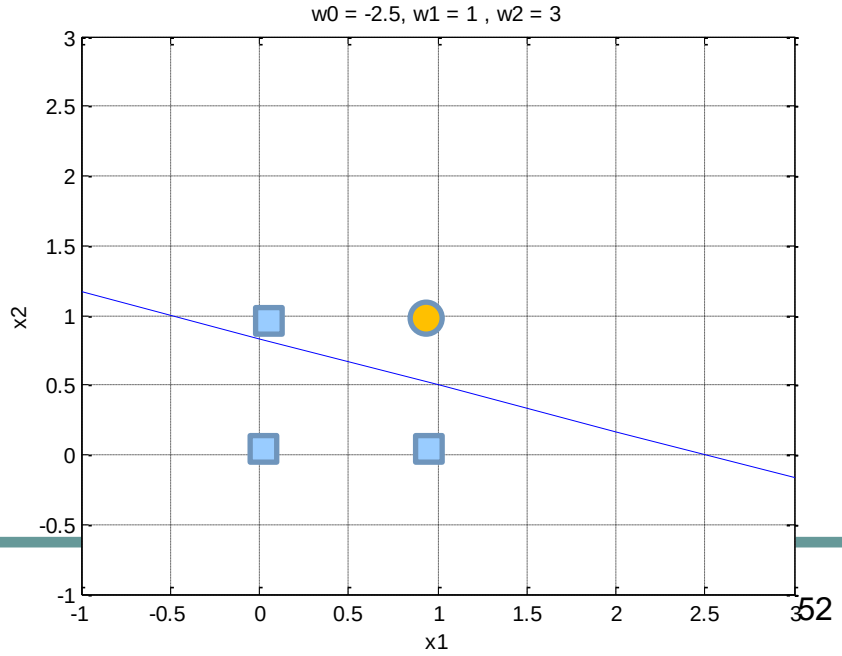
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no
1	0	1	0	-0.5	2	4	3.5	POS	yes
1	1	0	0	-1.5	2	3	0.5	POS	yes
1	1	1	1	-2.5	1	3	1	POS	no



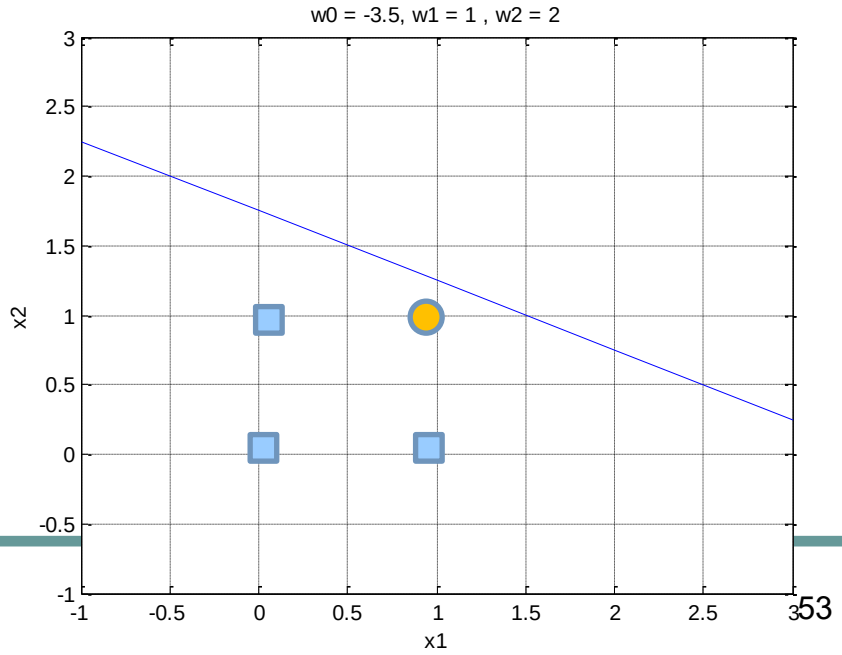
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	1	3	-2.5	NEG	no
1	0	1	0	-2.5	1	3	0.5	POS	yes
1	1	0	0	-3.5	1	2	-2.5	no	no



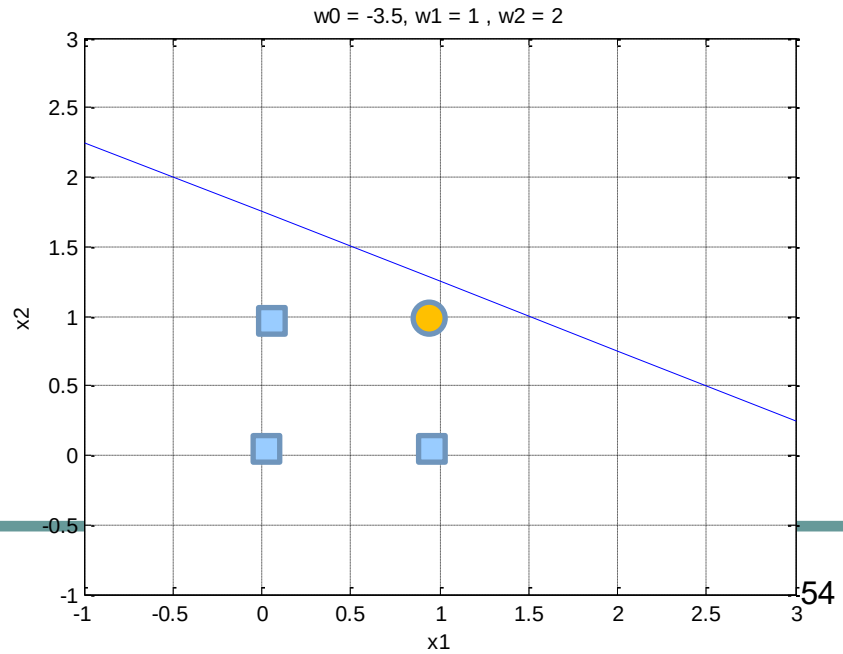
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	1	3	-2.5	NEG	no
1	0	1	0	-2.5	1	3	0.5	POS	Yes
1	1	0	0	-3.5	1	2	-2.5	NEG	No
1	1	1	1	-3.5	1	2	-0.5	NEG	yes



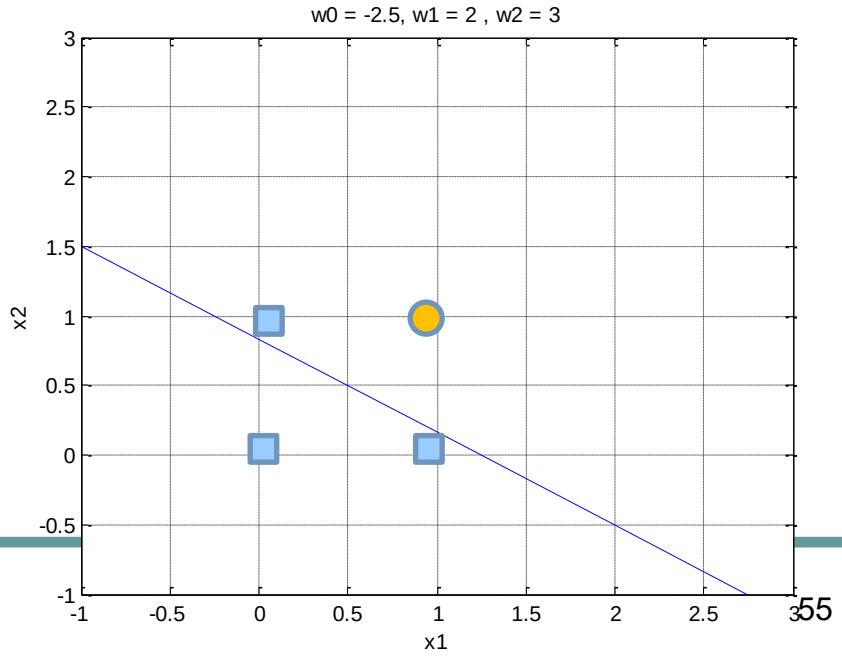
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	2	3	-3.5	NEG	no
1	0	1	0	-2.5	2	3	0.5	POS	yes
1	1	0	0						
1	1	1	1						



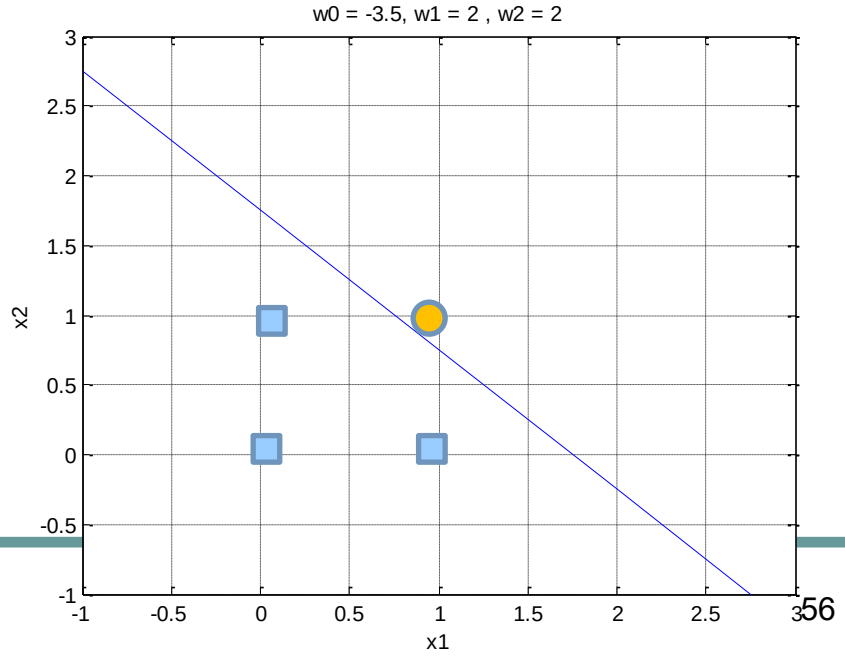
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

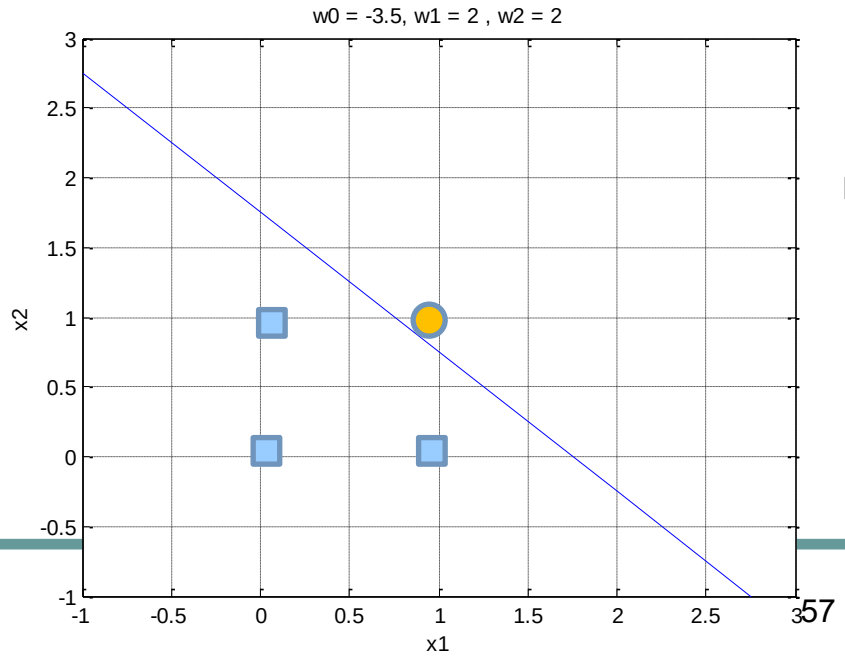
AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	2	3	-3.5	NEG	no
1	0	1	0	-2.5	2	3	0.5	POS	yes
1	1	0	0	-3.5	2	2	-1.5	NEG	no
1	1	1	1	-3.5	2	2	0.5	POS	no



AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-3.5	2	2	-3.5	NEG	no
1	0	1	0	-3.5	2	2	-1.5	NEG	no
1	1	0	0	-3.5	2	2	-1.5	NEG	no
1	1	1	1	-3.5	2	2	0.5	POS	no



נמשיך בתהליך עד
שנגיע ל"סיבוב" שלם
ללא צורך בעידכון

משפט ההתכנסות של פרספטרון

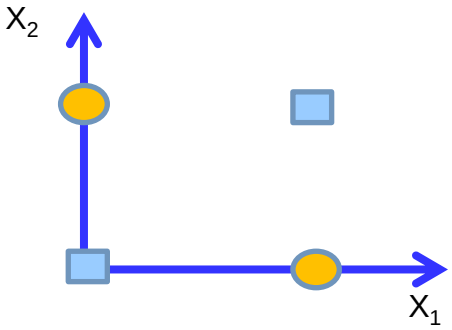
- הנח ששתי המחלקות N,P ניתנות להפרדה לינארית ע"י סט המשקולות w* (כלומר ישנו על-מישור hyperplane המסוגל להפריד ביניהם), אזי אלגוריתם הפרספטרון יתכנס לאחר מס' צעדים סופי לסט משקולות w' (לא בהכרח שקול ל-w*) המפריד בין שתי המחלקות.

Perceptron

- פרספטרון (בעל שכבת כניסה אחת ושכבת יציאה אחת, ללא שכבות חבויות) יכול לסווג נכון רק בעיות הניתנות להפרדה לינארית.
  - AND ניתן להפרדה לינארית
  - XOR לא ניתן להפרדה לינארית
- רשת נוירונים עם שכבות חבויות ופונקציית "הפעלה" יכולה ללמוד ולסווג בעיות מורכבות יותר (שאינן ניתנות להפרדה לינארית)

XOR problem

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

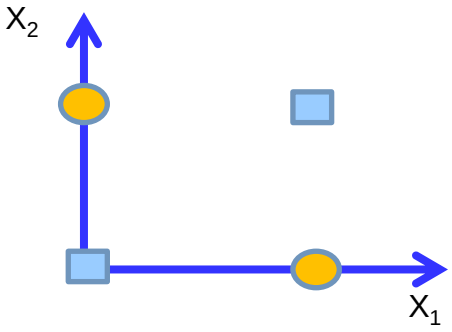


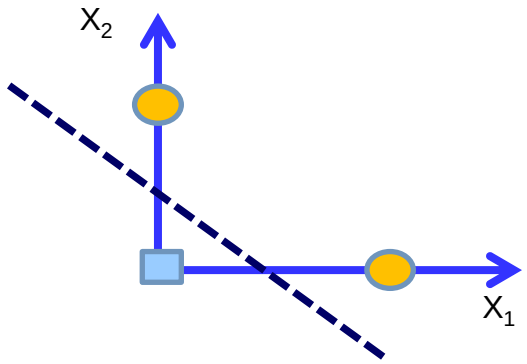
XOR problem

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

ייצוג שונה של המאפיינים

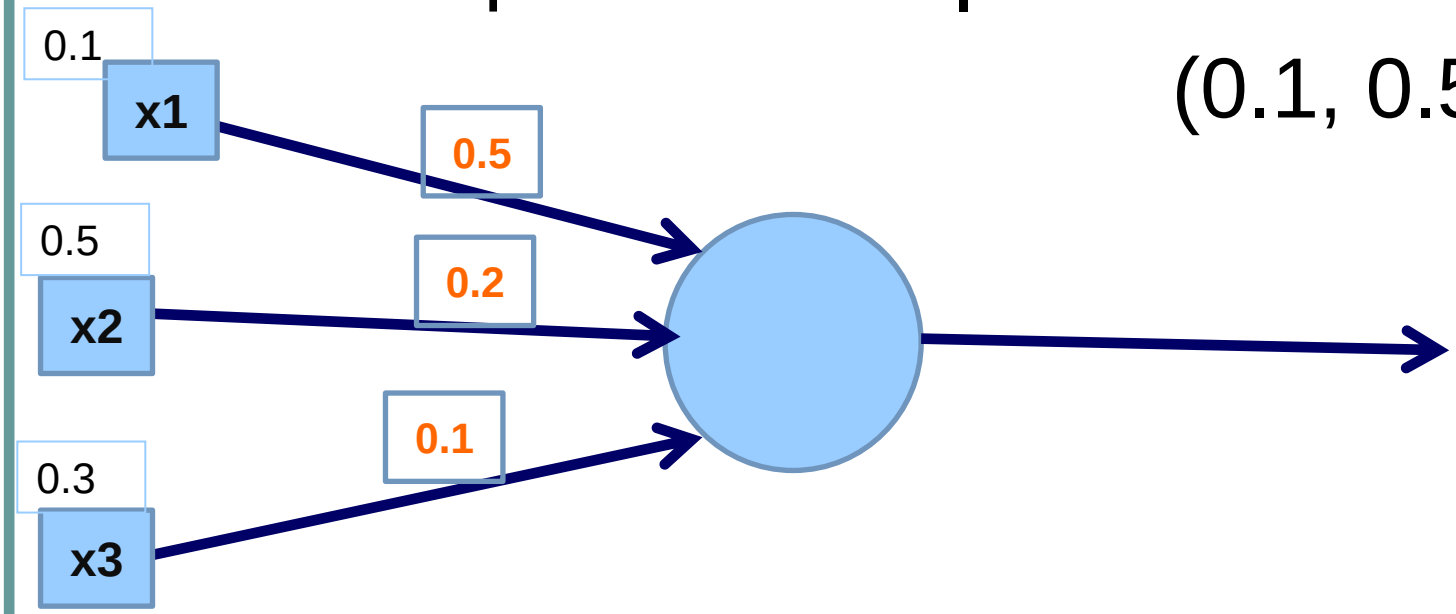
$AND(\overline{x1}, x2)$	$AND(x1, \overline{x2})$	Y
0	0	0
1	0	1
0	1	1
0	0	0





תרגיל

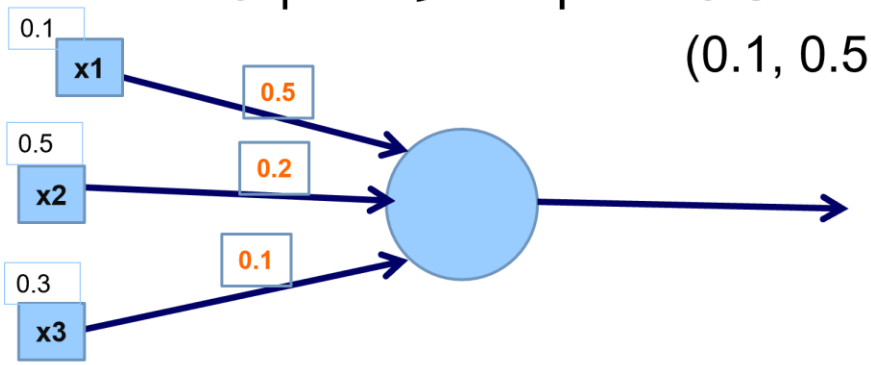
- חשב את פלט הנוירון הבא עבור הקלט:
(0.1, 0.5, 0.3)



1. עבור סף של 0
2. עבור פונקציית הסיגמואיד

פתרון - Threshold = 0

- חשב את פלט הנוירון הבא עבור הקלט:
  (0.1, 0.5, 0.3)



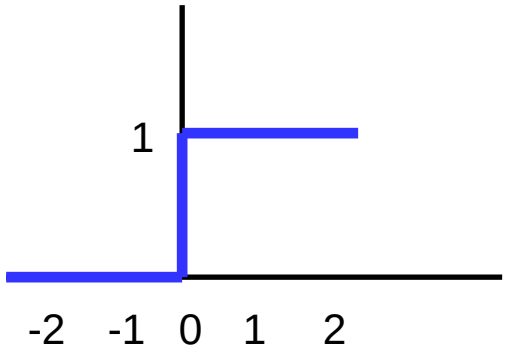
Sum =

- (0.1 x 0.5) + (0.5 x 0.2) + (0.3 x 0.1) = 0.05 + 0.1 + 0.03 = 0.18
- Activation with Threshold 0 (0.18> 0) → **output = 1**

דוגמאות לפונקציות אקטיבציה

SIGN

$$\operatorname{sgn}(x) = \begin{cases} 1 \text{ if } x \geq 0 \\ 0 \text{ if } x < 0 \end{cases}$$

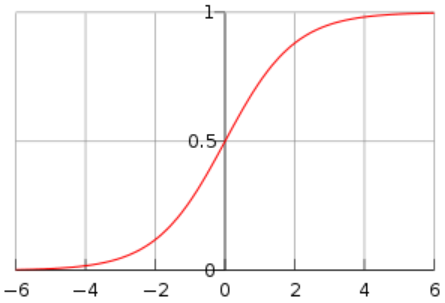


SIGMOID

פונקציה רציפה עם ערך בין 0 ל-1, תמיד "חיובית", חסומה

"squashing function" – The sigmoid (aka logistic) function is used to compress the outputs of the "neurons" in multi-layer neural nets

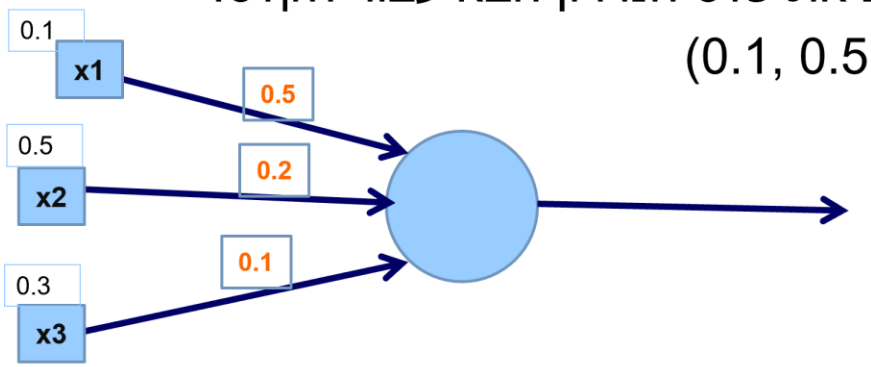
$$sigmoid(x) = \frac{1}{1+e^{-x}}$$



פתרון - Sigmoid

$$sigmoid(x) = \frac{1}{1+e^{-x}}$$

- חשב את פלט הנוירון הבא עבור הקלט:
  (0.1, 0.5, 0.3)

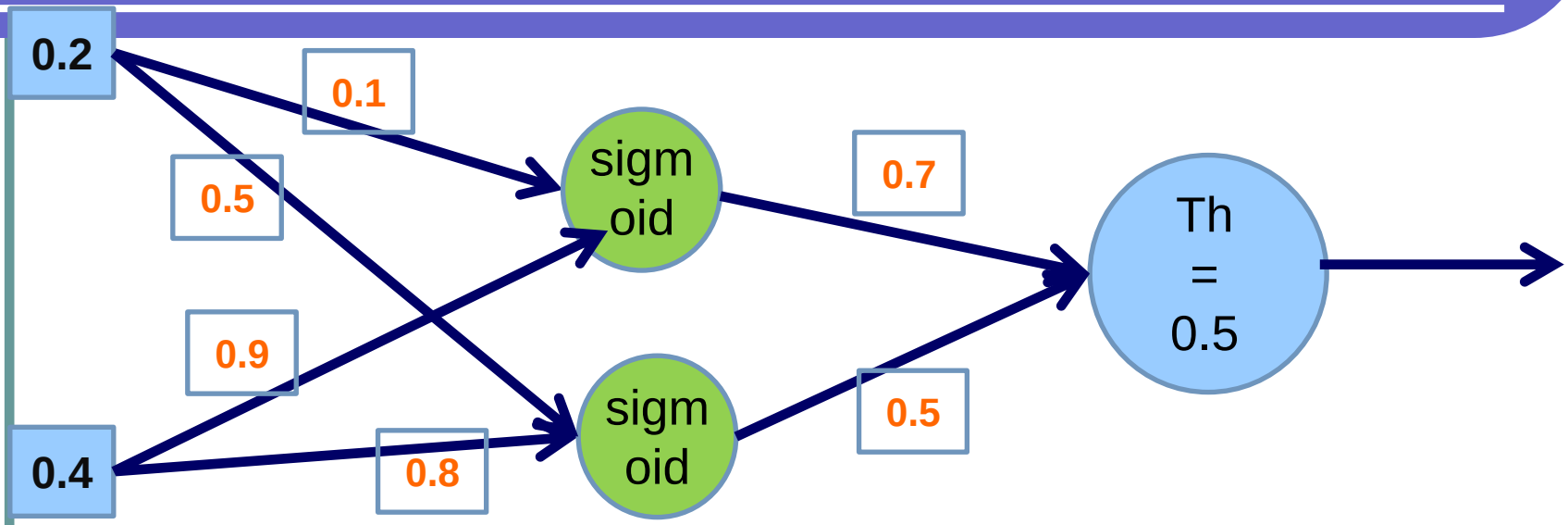


$$output = \frac{1}{1+e^{-0.18}} = 0.54$$

עם פונקציית ה-SIGMOID נוירון היציאה בעצם "חוזה" לנו את p(y=1|**x**)

- את הסיווג עצמו ניתן לבצע:
  - אם הערך ביציאה גדול/שווה מ-0.5 ← predict POS
  - אחרת, predict NEG

תרגיל



(0.2 x 0.5) + (0.4 x 0.8) = 0.42 $output = \frac{1}{1+e^{-0.42}} = 0.603$

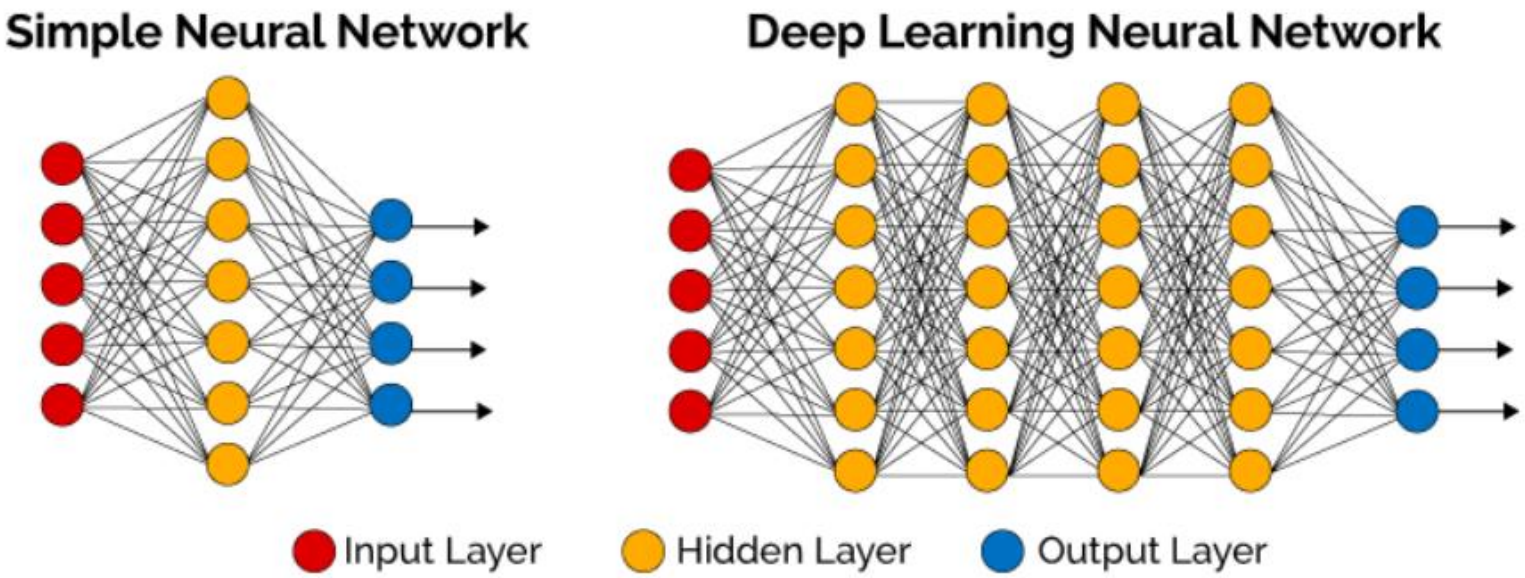
(0.2 x 0.1) + (0.4 x 0.9) = 0.38 $output = \frac{1}{1+e^{-0.38}} = 0.594$

(0.594 x 0.7) + (0.603 x 0.5) = 0.717 > 0.5 → output = 1

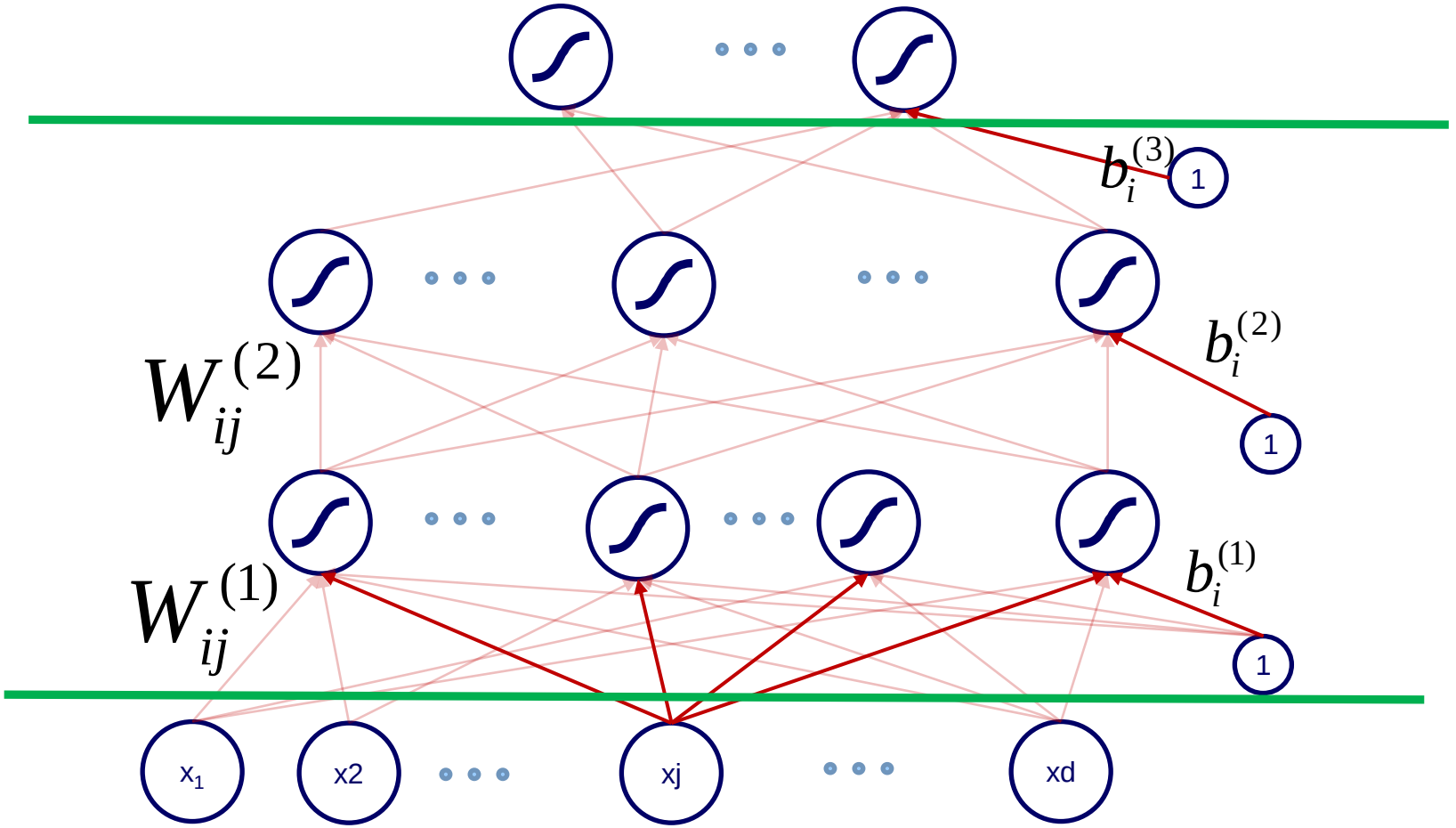
רשת נוירונים בעלת שכבות חבויות (MultiLayer Perceptron - MLP) - FeedForward

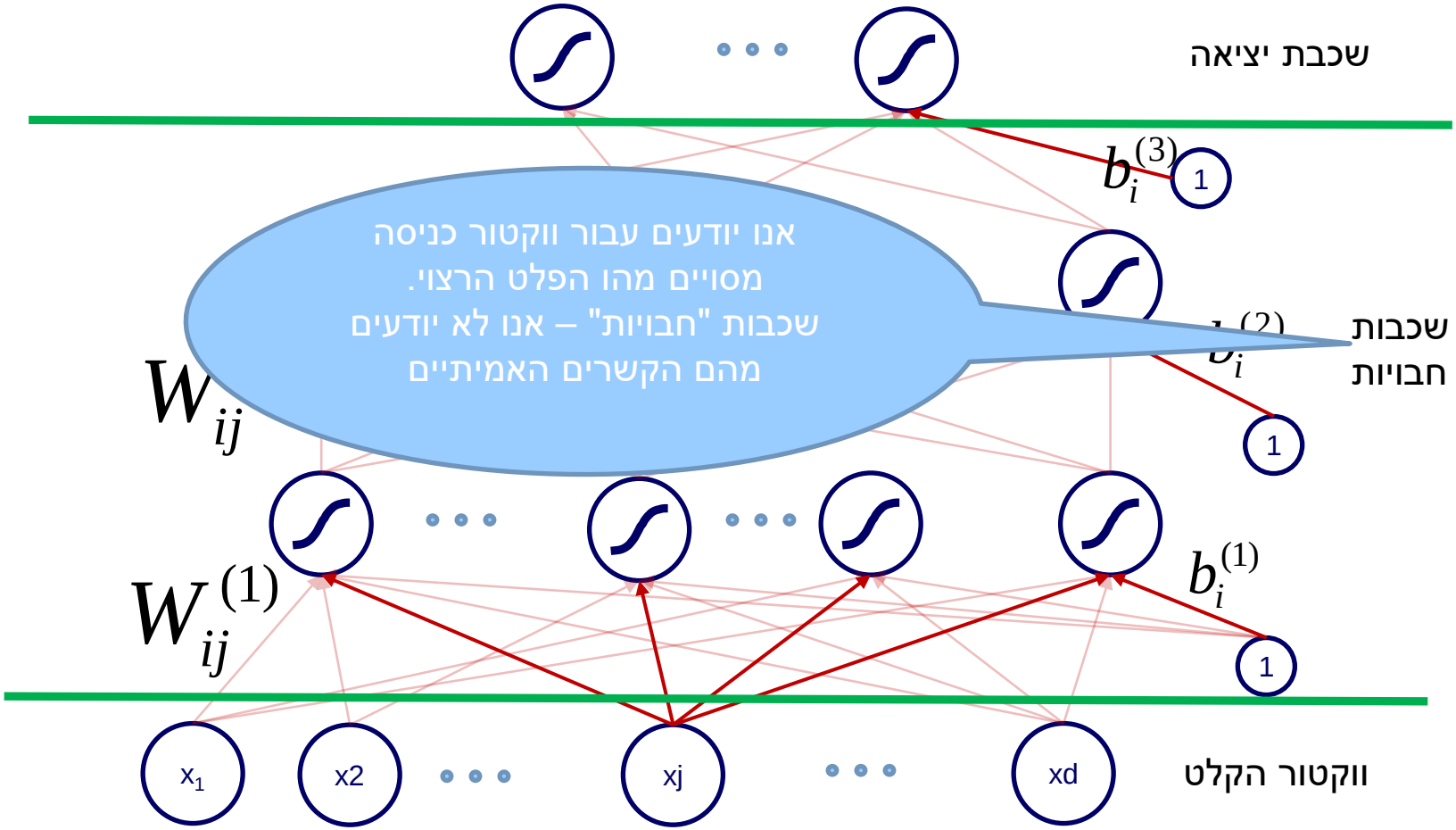
- "מס' שכבות" של נוירונים מסוג פרספטרון ,כאשר כל שכבה מזינה את השכבה הבאה אחריה.
- אין היזון חוזר לשכבות הקודמות .
- ללא שכבות חבויות – פרספטרון – הפרדה לינארית בלבד
- עם שכבות חבויות – ניתן להגיע להפרדה לא לינארית

Neural Network רשת נוירונים

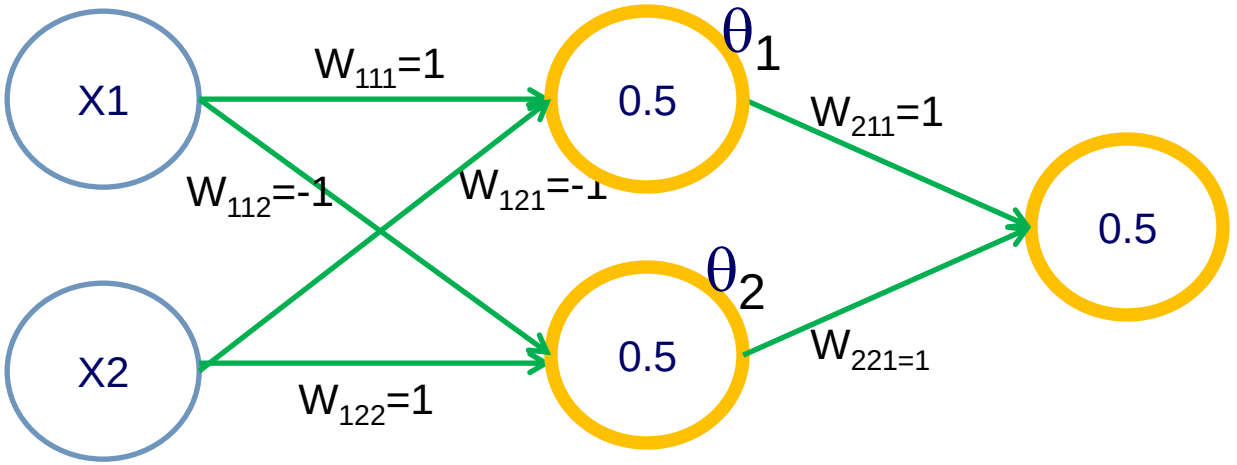


https://becominghuman.ai/deep-learning-made-easy-with-deep-cognition-403fbe445351


$b_i^{(3)}$
1
$W_{ij}^{(2)}$
$b_i^{(2)}$
1
$W_{ij}^{(1)}$
$b_i^{(1)}$
1
x_1
x2
xj
xd

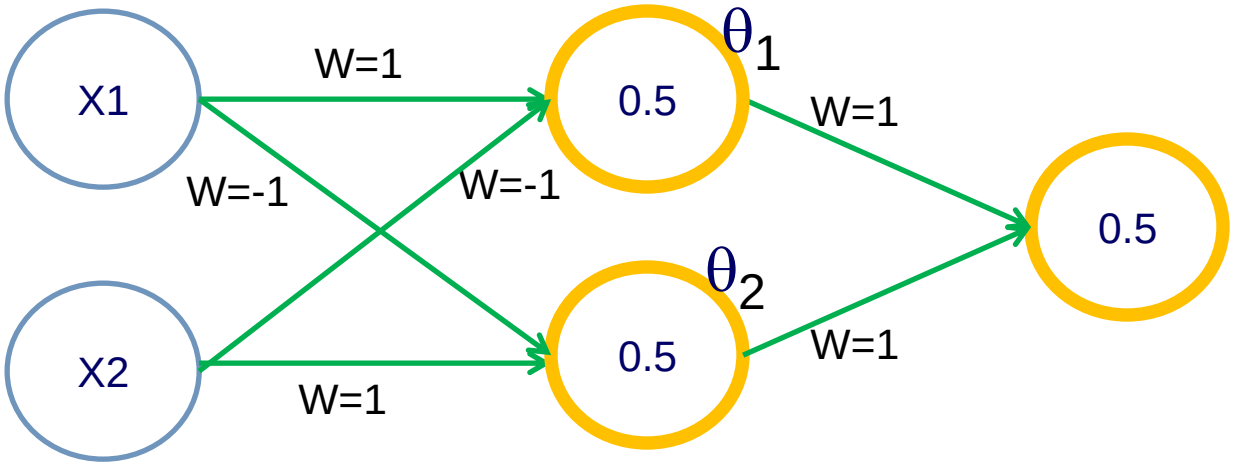

שכבת יציאה
$b_i^{(3)}$
1
אנו יודעים עבור ווקטור כניסה
מסויים מהו הפלט הרצוי.
שכבות "חבויות" – אנו לא יודעים
מהם הקשרים האמיתיים
שכבות
חבויות
W_{ij}
$b_i^{(2)}$
1
$W_{ij}^{(1)}$
$b_i^{(1)}$
1
x_1
x2
xj
xd
ווקטור הקלט

דוגמא לרשת עם שכבה חבויה אחת



$$out(x_1, x_2) = \text{sgn}(1 \times \theta_1 + 1 \times \theta_2 - 0.5)$$

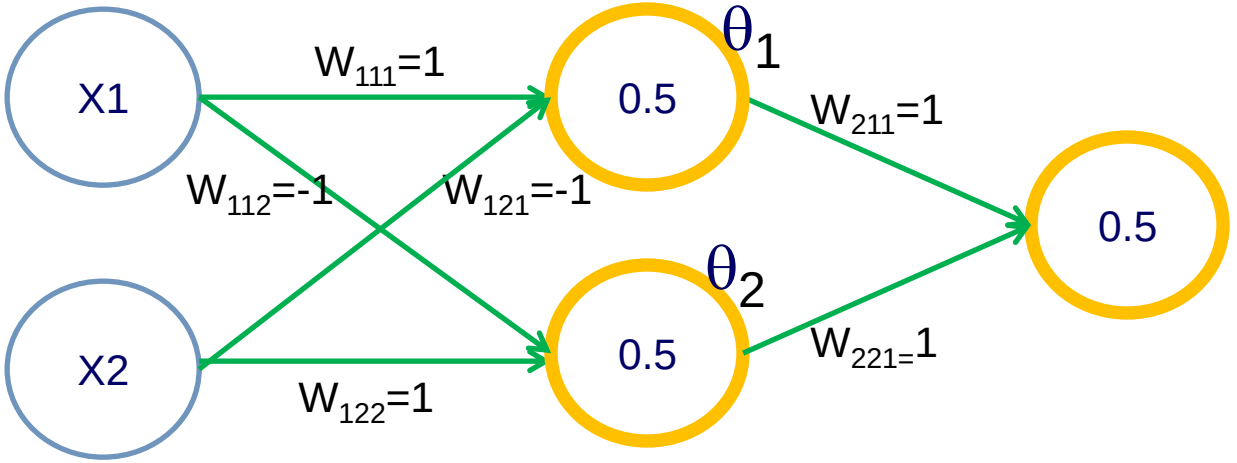
דוגמא לרשת עם שכבה חבויה אחת



$$\theta_1 = \text{sgn}(1 \times x_1 - 1 \times x_2 - 0.5)$$

$$\theta_2 = \text{sgn}(-1 \times x_1 + 1 \times x_2 - 0.5)$$

דוגמא לרשת עם שכבה חבויה אחת



$$out(x_1, x_2) =$$

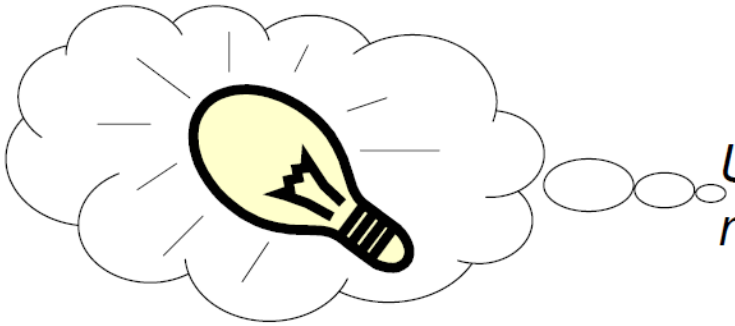
$$= \mathrm{sgn}(w_{211} \times \mathrm{sgn}(w_{111} \times x_1 + w_{121} \times x_2 - 0.5) + w_{221} \times \mathrm{sgn}(w_{112} \times x_1 + w_{122} \times x_2 - 0.5) - 0.5)$$

ניתן לבטא את פלט הרשת כפונקציה של w,x, ואז ניתן כמובן גם לגזור עבור פונקציות גזירות

Multilayer network

The class of functions representable by perceptrons is limited

$$\text{Out}(\mathbf{x}) = g\left(\mathbf{w}^{\mathrm{T}}\mathbf{x}\right) = g\left(\sum_j w_j x_j\right)$$



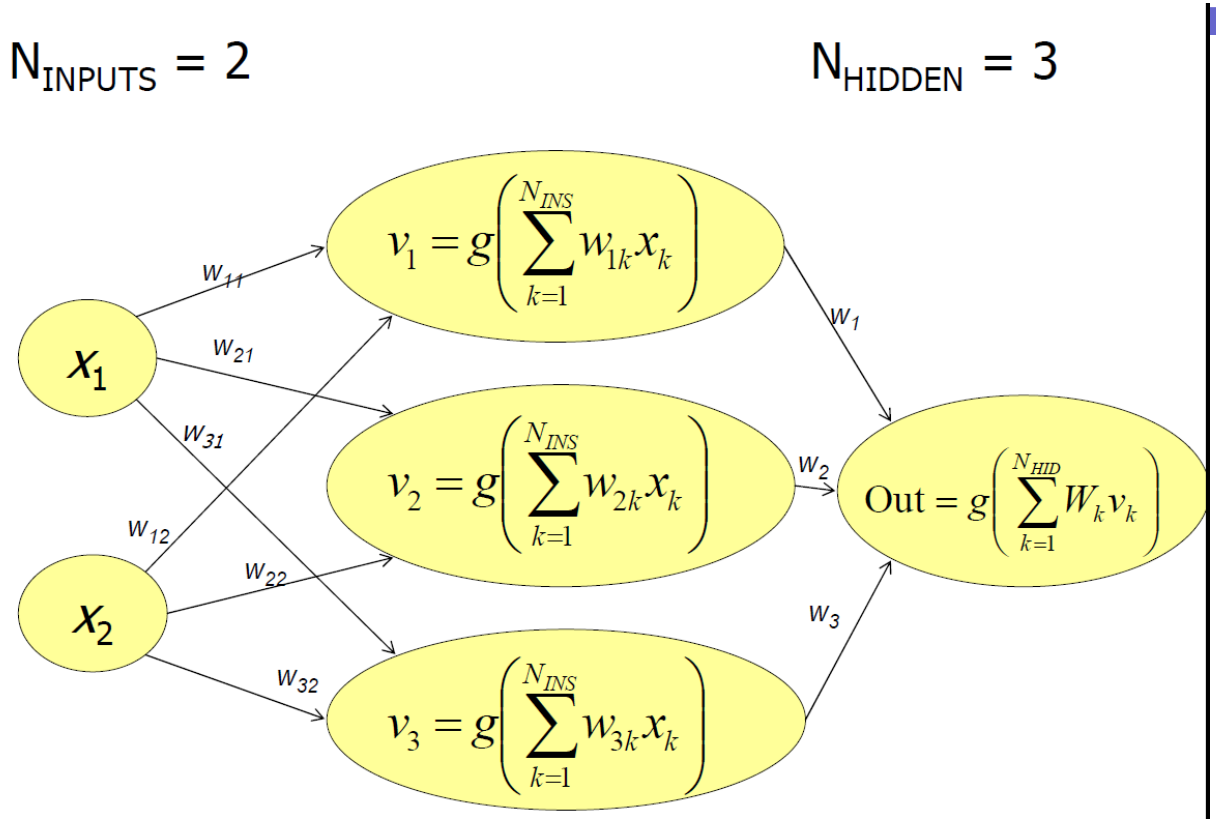
Use a wider representation !

$$\text{Out}(\mathbf{x}) = g\left(\sum_j W_j g\left(\sum_k w_{jk} x_{jk}\right)\right)$$

This is a nonlinear function
Of a linear combination
Of non linear functions
Of linear combinations of inputs

http://www.autonlab.org/tutorials/

רשת עם שכבה "חבויה" אחת

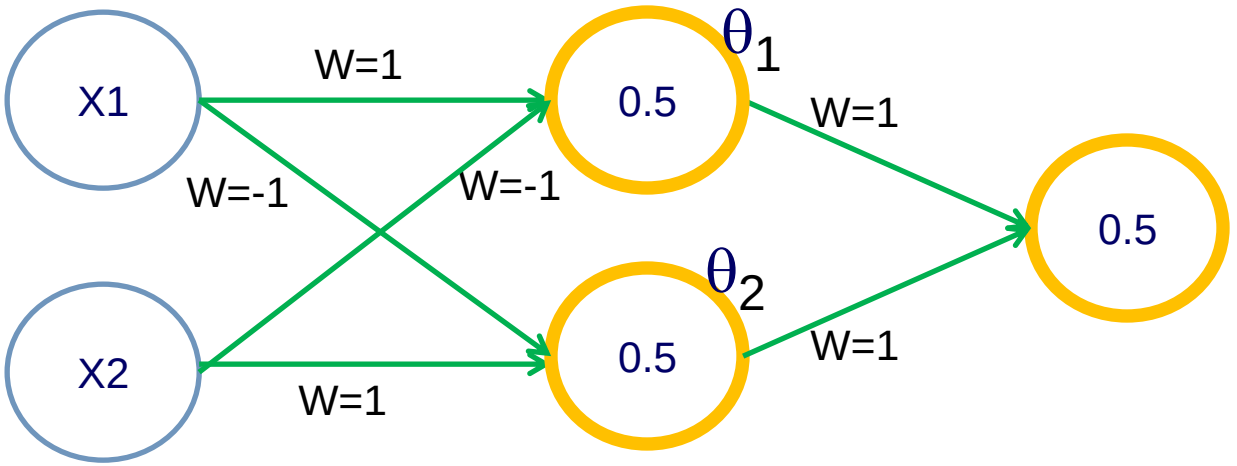


כדי להגיע ליכולת הפרדה לא לינארית חייבים להשתמש בפונקציה g שאינה לינארית

http://www.autonlab.org/tutorials/

XOR

XOR		
X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

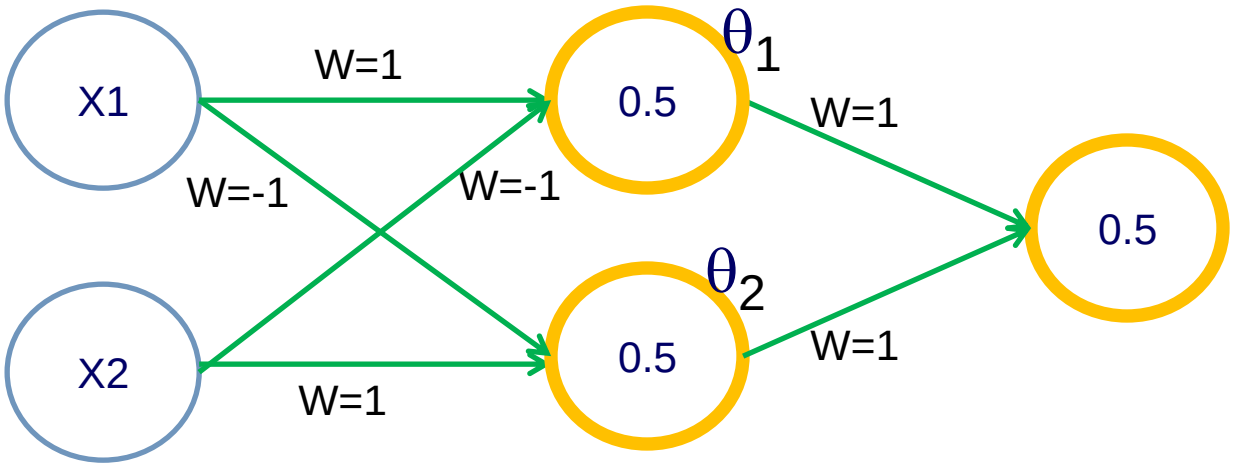


$$out(x_1, x_2) = \text{sgn}(1\times\theta_1 + 1\times\theta_2 - 0.5)$$
$$= \text{sgn}(1\times\text{sgn}(1\times x_1 - 1\times x_2 - 0.5) + 1\times\text{sgn}(-1\times x_1 + 1\times x_2 - 0.5) - 0.5)$$

$$out(1,1) = \text{sgn}(1\times\text{sgn}(1\times x_1 - 1\times x_2 - 0.5) + 1\times\text{sgn}(-1\times x_1 + 1\times x_2 - 0.5) - 0.5)$$
$$= \text{sgn}(1\times\text{sgn}(1\times 1 - 1\times 1 - 0.5) + 1\times\text{sgn}(-1\times 1 + 1\times 1 - 0.5) - 0.5)$$
$$= \text{sgn}(1\times\text{sgn}(-0.5) + 1\times\text{sgn}(-0.5) - 0.5)$$
$$= \text{sgn}(1\times 0 + 1\times 0 - 0.5) = 0$$

XOR

XOR		
X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0



$$out(x_1, x_2) = \text{sgn}(1\times\theta_1 + 1\times\theta_2 - 0.5)$$
$$= \text{sgn}(1\times\text{sgn}(1\times x_1 - 1\times x_2 - 0.5) + 1\times\text{sgn}(-1\times x_1 + 1\times x_2 - 0.5) - 0.5)$$

$$out(1,0) = \text{sgn}(1\times\text{sgn}(1\times x_1 - 1\times x_2 - 0.5) + 1\times\text{sgn}(-1\times x_1 + 1\times x_2 - 0.5) - 0.5)$$
$$= \text{sgn}(1\times\text{sgn}(1\times 1 - 1\times 0 - 0.5) + 1\times\text{sgn}(-1\times 1 + 1\times 0 - 0.5) - 0.5)$$
$$= \text{sgn}(1\times\text{sgn}(0.5) + 1\times\text{sgn}(-1.5) - 0.5)$$
$$= \text{sgn}(1\times 1 + 1\times 0 - 0.5) = 1$$